

GPan-LoRA: Gaussian Process Amortized Networks for Bayesian Low-Rank Adaptation in Large Language Models

Weifeng Zhang^{*1} Wenyuan Zhao^{*1} Amir Hossein Rahmati¹ Yucheng Wang¹
Zhiyuan Wang¹ Chao Tian¹ Xiaoning Qian^{1 2 3}

Abstract

Principled uncertainty quantification (UQ) is increasingly recognized as essential for trustworthy artificial general intelligence (AGI). Bayesian Low-Rank Adaptation (LoRA) provides a principled mechanism for uncertainty-aware fine-tuning of large language models (LLMs). However, existing techniques either face scalability constraints, e.g. Laplace-LoRA, or rely on approximate inference schemes that lead to poorly calibrated posterior uncertainty, often manifesting as overconfident predictions under distribution shift. To address this challenge, we propose GPan-LoRA, the first scalable Gaussian Process (GP)-based framework for Bayesian LoRA, which integrates neural network-based sparse GP approximations with amortized variational inference. By preserving the Bayesian function prior and posterior semantics intrinsic to GPs, GPan-LoRA achieves a faithful balance between computational scalability and principled UQ. Empirically, GPan-LoRA produces well-calibrated uncertainty that remains reliable under distribution shift, mitigating overconfident failures while preserving competitive task performance.

1. Introduction

Large Language Models (LLMs), such as GPT, Gemini, and their successors, have demonstrated remarkable capabilities in understanding and generating human-like text across vari-

^{*}Equal contribution ¹Department of Electrical & Computer Engineering, Texas A&M University, College Station, Texas, USA ²Department of Computer Science & Engineering, Texas A&M University, College Station, Texas, USA ³Applied Mathematics, Computing & Data Sciences, Brookhaven National Laboratory, Upton, New York, USA. Correspondence to: Weifeng Zhang <weifengzhang@tamu.edu>, Wenyuan Zhao <wyzhao@tamu.edu>.

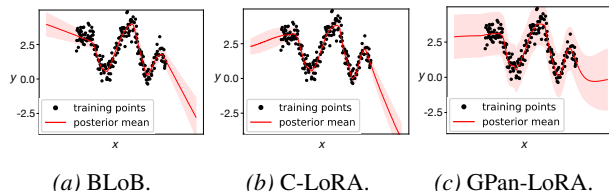


Figure 1. Fine-tuning a regression dataset with Bayesian LoRA. BLoB and C-LoRA exhibit low empirical coverage (red shadow) of confidence intervals, indicating overconfidence, while GPan-LoRA produces well-calibrated uncertainty estimates.

ous tasks (Wei et al., 2022; Teubner et al., 2023; Zhao et al., 2023; Chang et al., 2024; Majumder et al., 2024). However, the inherent uncertainty (Abdar et al., 2021) in their outputs poses significant challenges to reliable deployment in critical applications, such as healthcare, legal advisory, finance, and autonomous systems (Kasneci et al., 2023; Kaddour et al., 2023; Thirunavukarasu et al., 2023).

Bayesian learning naturally provides uncertainty quantification (UQ) by modeling predictive $P(\mathbf{y}|\mathbf{x}, \mathcal{D})$, which is the marginalization of posterior $P(\theta|\mathcal{D})$:

$$P(\mathbf{y}|\mathbf{x}, \mathcal{D}) = \int P(\mathbf{y}|\mathbf{x}, \theta)P(\theta|\mathcal{D})d\theta. \quad (1)$$

Exact posterior inference in LLMs is computationally infeasible due to their massive scale, making it challenging to accurately estimate uncertainty.

Low-rank adaptation (LoRA, Hu et al., 2022) is one of the most widely used parameter-efficient fine-tuning (PEFT) for LLMs. Recent research explores Bayesian inference under the LoRA mechanism to calibrate LLMs (Wang et al., 2023; Yang et al., 2023; Onal et al., 2024). For example, Bayesian Low-rank adaptation by Backpropagation (BLoB, Wang et al., 2024) jointly estimates the variational posterior of Bayesian neural networks (BNNs) throughout the entire fine-tuning stage. C-LoRA (Rahmati et al., 2025) further improves scalability with amortized variational low-rank parameters, allowing sample-dependent uncertainty.

Existing Bayesian LoRA models face a fundamental trade-off between scalability and UQ quality: Approaches that provide well-calibrated and expressive uncertainty estimates

often incur prohibitive computational costs (Balabanov & Linander, 2024), while scalable models typically rely on simplified or heuristic uncertainty representations that compromise UQ fidelity (Gal & Ghahramani, 2016; Boluki et al., 2020; Fan et al., 2021). As illustrated in Figure 1, when evaluated on a toy regression task, BLoB exhibits the issue of overconfidence during fine-tuning. Even with sample-dependent amortization, C-LoRA still struggles with overconfidence.

Gaussian processes (GPs), as a principled probabilistic approach, have been successfully combined with neural network (NN) heads in deep kernel learning (DKL, Wilson et al., 2016a;b). Similar overfitting trends were identified by Ober et al. (2021) in DKL, noting that while fully Bayesian models can help alleviate overfitting, they are not feasible for LLMs due to their extremely high computational and memory requirements. More recently, Matias et al. (2024) proposed amortized variational DKL (AVDKL) to mitigate the overfitting problem by using amortized inducing points and a parameter-sharing scheme.

Despite the strong UQ guarantees, integrating GP-based Bayesian modeling into PEFT frameworks such as LoRA for LLMs is highly non-trivial (Bradshaw et al., 2017). First, classical GPs scale poorly with both the number of data points N and the input dimensionality D (Williams & Rasmussen, 2006), while LLMs operate in regimes where both are extremely large. Although sparse and approximate GP methods alleviate the cubic dependence on N and LoRA reduces the dimensionality to low rank r , applying sparse GPs directly to LoRA layers effectively results in very deep GPs (DGPs, Damianou & Lawrence, 2013), significantly increasing the complexity further (Salimbeni & Deisenroth, 2017). Second, classical GP inference is based on kernel matrix operations, including matrix inversion or linear solves (Titsias, 2009), which are fundamentally misaligned with the training and inference characteristics of LLMs. As a result, naively combining GPs with LoRA across all layers is prohibitively expensive and incompatible with the efficiency requirements of modern LLM fine-tuning.

To address the above challenge, we propose a new Bayesian framework of **Low-Rank Adaptation by Gaussian Processes amortized networks (GPan-LoRA)**. A variational sparse approximation, together with amortized inference, is applied to low-rank GP, offering a favorable scalability–UQ capability trade-off compared to prior GP-based Bayesian formulations, while maintaining the strong UQ properties intrinsic to GPs. We summarize the following contributions:

1. We propose GPan-LoRA, a scalable GP-based framework for Bayesian LoRA that enables principled UQ in LLMs with low computational overhead.
2. GPan-LoRA provides an end-to-end Bayesian framework that estimates uncertainty jointly with model

Table 1. Qualitative comparison of UQ methods in LoRA variants, where scalability reflects parameter efficient model and UQ quality captures the calibrated predictive uncertainty (✓: strong, △: moderate, ✗: weak).

Method	UQ Quality	Scalability
MLE (Hu et al., 2022)	✗	✓
MCD (Gal & Ghahramani, 2016)	✗	✓
DeepEns (Wang et al., 2023)	△	✗
LAP (Yang et al., 2023)	✓	✗
BLoB (Wang et al., 2024)	△	△
C-LoRA (Rahmati et al., 2025)	△	✓
GPan-LoRA (ours)	✓	△

adaptation, mitigating the issue of overconfidence.

3. We demonstrate better UQ quality and scalability in our proposed GPan-LoRA across various LLM backbones and different use cases.

For reproducibility, our implementation is publicly available on Github¹.

2. Preliminaries

2.1. Bayesian LoRA

LoRA assumes a low-rank decomposition for the weight updates to fine-tune a pre-trained network (Hu et al., 2022). Specifically, instead of directly updating the full weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$ in a linear layer, LoRA approximates the surrogate weight $\Delta\mathbf{W}$ as the product of two low-rank matrices $\mathbf{B} \in \mathbb{R}^{m \times r}$ and $\mathbf{A} \in \mathbb{R}^{r \times n}$:

$$\mathbf{h} = (\mathbf{W}_0 + \Delta\mathbf{W})\mathbf{x} = \mathbf{W}_0\mathbf{x} + \mathbf{B}\mathbf{A}\mathbf{x}, \quad (2)$$

where $r \ll \min\{m, n\}$. The original pre-trained weights $\mathbf{W}_0$ are kept frozen during fine-tuning, and only the low-rank matrices $\mathbf{A}$ and $\mathbf{B}$ are optimized.

Bayesian LoRA variants introduce probabilistic priors over LoRA weights and perform variational inference (VI) on posterior, enabling uncertainty-aware PEFT directly in the adaptation layers (Yang et al., 2023). BLoB (Wang et al., 2024), as one of Bayesianized LoRA frameworks, simultaneously updates the mean and covariance of $\Delta\mathbf{W}$ through Variational Bayesian Networks (VBN) with *asymmetric* Bayesian weights of LoRA: $q_\theta(\mathbf{A}) = \mathcal{N}(\mathbf{A} | \mathbf{M}_A, \Omega_A^2)$ and $q(\mathbf{B}) = \mathcal{N}(\mathbf{B} | \mathbf{0}, \Omega_B^2)$ with $\Omega_B \rightarrow \mathbf{0}^+$. The exact posterior $p(\mathbf{A} | \mathcal{D})$ is generally intractable, and the variational posterior $q_\theta(\mathbf{A})$ is learned through VI, which minimizes $\text{KL}[q_\theta(\mathbf{A}) \| p(\mathbf{A} | \mathcal{D})]$. The training objective is equivalent to minimizing the loss of variational free energy (VFE) with respect to variational parameters $\theta = \{\mathbf{M}_A, \Omega_A^2\}$:

$$\mathcal{L}(\theta) = -\mathbb{E}_{q_\theta} [\log P(\mathcal{D} | \mathbf{A})] + \text{KL}[q_\theta(\mathbf{A}) \| p(\mathbf{A})], \quad (3)$$

where $p(\mathbf{A}) := \mathcal{N}(\mathbf{A} | \mathbf{0}, \sigma^2 \mathbf{I})$ is the prior of $\mathbf{A}$.

¹<https://github.com/WEIFZH/GPan-LoRA>

2.2. GPs and Amortized VI

A GP $f(\cdot) \sim \mathcal{GP}(\mu, k)$ is fully specified by the mean function $\mu(\cdot)$ and the kernel function (covariance) $k(\cdot, \cdot)$. Let $\mathbf{x} \in \mathbb{R}^n$ denote a feature representation. We consider a Q -dimensional prediction head

$$\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_Q(\mathbf{x})), \quad (4)$$

modeled by *independent* GPs,

$$f_q(\mathbf{x}) \sim \mathcal{GP}(0, k(\mathbf{x}, \mathbf{x}')), \quad q = 1, \dots, Q, \quad (5)$$

sharing a common kernel k . The computational cost of exact GPs grows cubically with the number of training points, $O(N^3)$, making them unsuitable for use on large datasets. For more details, we direct readers to Williams & Rasmussen (2006).

For scalability, sparse variational GPs (Titsias, 2009) approximate each output associated with $M \ll N$ inducing variables $\mathbf{u}_q = f_q(\mathbf{Z})$, $\mathbf{Z} = \{\mathbf{z}_m\}_{m=1}^M$, with shared inputs across tasks. The variational posterior factorizes across Q outputs: $q(\mathbf{u}) = \prod_{q=1}^Q \mathcal{N}(\mathbf{m}_q, \mathbf{S}_q)$.

Instead of optimizing $\theta := \{(\mathbf{m}_q, \mathbf{S}_q)\}_{q=1}^Q$ with VFE loss directly, we perform amortized VI (AVI) with a lightweight inference network f_ϕ^{inf} conditioned on hidden features:

$$\{(\mathbf{m}_q, \mathbf{S}_q)\}_{q=1}^Q = f_\phi^{\text{inf}}(\mathbf{x}), \quad q_\phi(\mathbf{u} | \mathbf{x}). \quad (6)$$

This enables variational parameters that depend on the input while still permitting efficient inference, and it also mitigates overconfidence, as discussed by Matias et al. (2024).

3. Related Work

3.1. Bayesian LoRA

Standard PEFT methods are deterministic and do not provide principled uncertainty estimates over task-specific adaptations. Recent work has begun to explicitly Bayesianize LoRA mechanisms for LLMs. Laplace-LoRA (LAP, Yang et al., 2023) introduces a probabilistic treatment of LoRA parameters, demonstrating that uncertainty-aware LoRA can improve robustness with calibration. Similarly, BLoB (Wang et al., 2024) is a variational approach that jointly optimizes the mean and covariance of LoRA parameters throughout the fine-tuning process, which leads to improved generalization and uncertainty estimation on both in-distribution and out-of-distribution data. Training-Free Bayesianization (TFB) (Shi et al., 2024) further develops BLoB by transforming existing off-the-shelf LoRA adapters into isotropic Gaussian distributions, offering maximally acceptable variance across deterministic LoRA backbones. C-LoRA (Rahmati et al., 2025) introduces an end-to-end Bayesian LoRA framework with contextual modules, allowing sample-dependent uncertainty.

3.2. GPs and UQ methods

GPs provide principled Bayesian function priors with well-calibrated uncertainty, and their close connection to NNs has long been established, with infinitely wide networks converging to GPs and kernel-based interpretations of deep models (Neal, 1996; Williams, 1996; Jacot et al., 2018). DKL further combines neural feature extractors with GPs to improve expressiveness, typically relying on sparse inducing approximations for scalability (Wilson et al., 2016a; Hensman et al., 2013). However, directly applying GP-based methods to LoRA models is non-trivial: placing GPs at multiple LoRA layers naturally induces a deep GP, leading to stacked uncertainty propagation and prohibitive inference complexity (Salimbeni & Deisenroth, 2017).

There are many other UQ methods, including MC dropout (MCD, Gal & Ghahramani, 2016) and deep ensemble (Deep-Ens, Lakshminarayanan et al., 2017). As shown in Table 1, existing UQ methods on LoRA models still face a fundamental trade-off between scalability and UQ quality, especially regarding the overfitting problem when distribution shifts exist. GPan-LoRA distinguishes itself by confining GP inference to the low-rank feature space, thereby achieving an effective balance that maintains both scalability and rigorous uncertainty for LLMs.

4. GPan-LoRA: Methodology

We now formally introduce GPan-LoRA for Bayesian low-rank adaptation. We extend the linear structure in (2) to a more general form of the hidden layer in a function space by Gaussian Process amortized networks (GPan):

$$\mathbf{h} = f_0(\mathbf{x}) + f_\Delta(\mathbf{x}), \quad (7)$$

where $f_0(\cdot)$ denotes the pretrained base model and is usually assumed to be $\mathbf{W}_0\mathbf{x}$ in LoRA. To facilitate uncertainty estimation, we employ GPs to represent the discrepancy between the outputs of the fine-tuned model and the pretrained model, $f_\Delta(\cdot) \sim \mathcal{GP}(\mu, k)$:

$$f_\Delta(\cdot) := g(\cdot) + \mu(\cdot), \quad (8)$$

where $g(\cdot) \sim \mathcal{GP}(0, k)$ is a *zero-mean* GP with covariance function $k(\cdot, \cdot)$, and $\mu(\cdot)$ is the mean function of $f_\Delta(\cdot)$.

To ensure scalability and efficiency, rather than directly solving $f_\Delta(\mathbf{x})$ in the high-dimensional space $\mathbf{x} \in \mathbb{R}^n$, we learn a low-rank adaptation of $g(\cdot)$ in the reduced space $\mathbf{A}\mathbf{x} \in \mathbb{R}^r$, formulated as a low-rank GP with amortized network, GPan-LoRA, as illustrated in Figure 2(c).

4.1. GP under low-rank feature maps

Let $g(\cdot) \sim \mathcal{GP}(0, k)$ be a GP on $\mathbf{x} \in \mathbb{R}^n$. Considering a low-dimensional linear transformation

$$\tau(\mathbf{x}) = \mathbf{A}\mathbf{x}, \quad \mathbf{A} \in \mathbb{R}^{r \times n}, \quad (9)$$

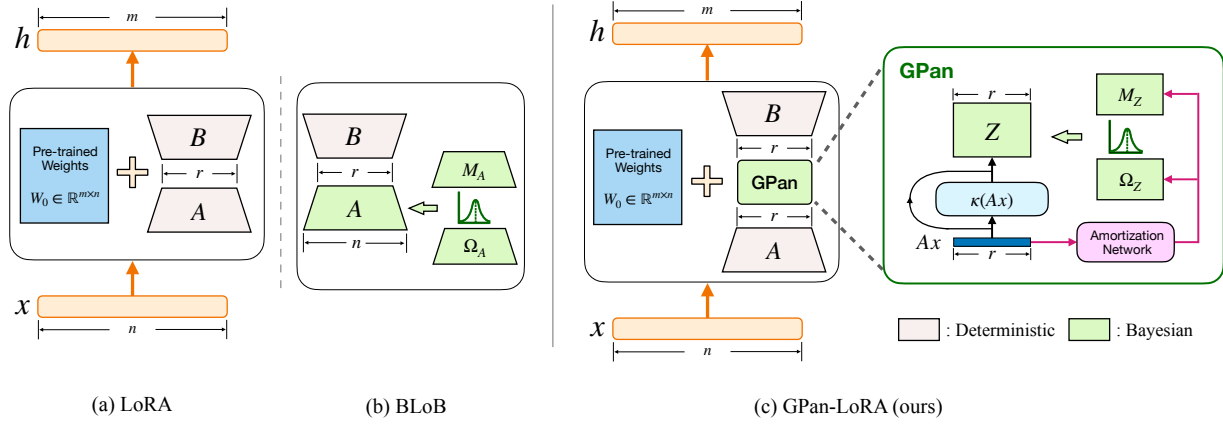


Figure 2. Module architectures of LoRA (left), BLoB (middle), and GPan-LoRA (right). While BLoB places Bayesian priors directly on LoRA factors, introducing Bayesian parameters that scale with the full input dimension $O(nr)$, GPan-LoRA confines Bayesian inference to the low-rank latent space, requiring only $O(lr^2)$ Bayesian variables via a low-rank GP amortized network.

we say that $g(\cdot)$ can be *exactly recast* through $\tau(\mathbf{x}) = \mathbf{Ax}$ if there exists a GP $\tilde{g}(\cdot) \sim \mathcal{GP}(0, \tilde{k})$ on $\mathbb{R}^r$ such that $\{g(\mathbf{x}) : \mathbf{x} \in \mathbb{R}^n\}$ and $\{\tilde{g}(\mathbf{Ax}) : \mathbf{Ax} \in \mathbb{R}^r\}$ have the same finite-dimensional distributions. Equivalently, the random function $g(\cdot)$ depends on $\mathbf{x}$ only through the projected coordinate $\tau(\mathbf{x}) = \mathbf{Ax}$.

Theorem 4.1 (Exact GP recasting through linear transformations). *The following statements are equivalent:*

- 1) There exists a zero-mean GP $\tilde{g}(\cdot)$ on $\mathbb{R}^r$ such that $g(\mathbf{x}) \stackrel{d}{=} \tilde{g}(\mathbf{Ax})$.
- 2) There exists a covariance function $\tilde{k}(\cdot, \cdot) : \mathbb{R}^r \times \mathbb{R}^r \rightarrow \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \tilde{k}(\mathbf{Ax}, \mathbf{Ax}')$.

Moreover, when these conditions hold, $\tilde{k}$ is positive semi-definite on the column space $\text{range}(\mathbf{A}) \equiv \{\mathbf{Ax} : \mathbf{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^r$ and therefore defines a valid GP.

Low-rank kernel. While Theorem 4.1 holds for arbitrary kernels, we focus on stationary kernels in this work, which are the most commonly used choices in GPs. In particular, the squared exponential (SE) kernel $k_{\text{SE}}(\mathbf{x}, \mathbf{x}')$ can be recast as an *isotropic* kernel π_{SE} with a low-rank lengthscale matrix $\Delta^{-1} = \mathbf{A}^T \mathbf{A}$ (de Souza et al., 2023) by applying a linear transformation $\tau(\mathbf{x}) = \mathbf{Ax} : \mathbb{R}^n \rightarrow \mathbb{R}^r$ to $\mathbf{x} \in \mathbb{R}^n$:

$$k_{\text{SE}}(\mathbf{x}, \mathbf{x}') = \sigma^2 \exp \left[-\frac{1}{2} (\mathbf{x} - \mathbf{x}')^\top \Delta^{-1} (\mathbf{x} - \mathbf{x}') \right] \quad (10)$$

$$= \sigma^2 \exp \left[-\frac{1}{2} \|\mathbf{Ax} - \mathbf{Ax}'\|^2 \right] \quad (11)$$

$$= \pi_{\text{SE}}(\mathbf{Ax}, \mathbf{Ax}'). \quad (12)$$

The corresponding $g(\cdot)$ is given by

$$\mathcal{GP}(0, \pi_{\text{SE}}(\mathbf{Ax}, \mathbf{Ax}')) \Leftrightarrow \mathcal{GP}(0, k_{\text{SE}}(\mathbf{x}, \mathbf{x}')). \quad (13)$$

4.2. Low-rank GP as a hidden layer of VBN

Simply merging low-rank adaptation with GPs does not *directly* yield a computationally efficient structure or an inference procedure that integrates well with LLMs, particularly in the context of a DGP model for maintaining the desired model capacity. To address the intensive computational complexity of DGPs, we approximate the GPs in each hidden layer with VBNs using the deep additive kernel.

VBN approximation of $g(\cdot)$. Let $\mathbf{u} \in \mathbb{R}^l$ represent the vector that collects the l inducing points associated with each GP head. $g(\cdot)$ can be represented approximately using a sparsely additive induced kernel (Zhao et al., 2025):

$$g(\mathbf{Ax}) \approx \sum_{i=1}^r k(\mathbf{a}_i^\top \mathbf{x}, \mathbf{u}) [k(\mathbf{u}, \mathbf{u})]^{-1} g(\mathbf{u}) \quad (14)$$

$$= \sum_{i=1}^r k(\mathbf{a}_i^\top \mathbf{x}, \mathbf{u}) [\mathbf{L}_u^\top]^{-1} \mathbf{L}_u^{-1} g(\mathbf{u}) \quad (15)$$

$$\triangleq \sum_{i=1}^r \mathbf{z}_i^\top \kappa(\mathbf{a}_i^\top \mathbf{x}, \mathbf{u}), \quad (16)$$

where $\mathbf{Ax} \in \mathbb{R}^r$ is the embedded low-rank feature, and $\mathbf{L}_u \in \mathbb{R}^{l \times l}$ is the Cholesky decomposition of $k(\mathbf{u}, \mathbf{u})$.

Equation (15) can be decomposed into two components: $\kappa(\cdot) = \mathbf{L}_u^{-1} k(\mathbf{u}, \cdot)$, which represents a nonlinear deterministic activation; and $\mathbf{z}_i = \mathbf{L}_u^{-1} g(\mathbf{u}) \in \mathbb{R}^l$, which is an i.i.d. Gaussian random vector. Therefore, the induced kernel approximation of $g(\cdot)$ in (16) is mathematically equivalent to a hidden layer of VBN.

After vectorization and concatenation of r independent GP heads, $f_\Delta(\mathbf{x})$ can ultimately be written in the matrix form

as a low-rank hidden GP layer:

$$f_{\Delta}(\mathbf{x}) := g(\mathbf{A}\mathbf{x}) + \mu(\mathbf{A}\mathbf{x}) \quad (17)$$

$$:= \mathbf{B}\mathbf{Z}\kappa(\mathbf{A}\mathbf{x}) + \mu(\mathbf{A}\mathbf{x}), \quad (18)$$

where $\mathbf{A} \in \mathbb{R}^{r \times n}$ and $\mathbf{B} \in \mathbb{R}^{m \times r}$ are *deterministic* matrices, and $\mathbf{Z} \in \mathbb{R}^{r \times lr}$ is a *random* matrix, enabling efficient joint Bayesian inference.

Bayesian residual mean $\mu(\cdot)$. To enhance expressive capacity, we impose a priori on the mean function $\mu(\mathbf{A}\mathbf{x})$ and follow the asymmetric LoRA Bayesianization

$$\mu(\mathbf{x}) := \mathbf{B}\Sigma\mathbf{A}\mathbf{x}, \quad (19)$$

where $\mathbf{A} \in \mathbb{R}^{r \times n}$ and $\mathbf{B} \in \mathbb{R}^{m \times r}$ are deterministic weight matrices shared with $g(\cdot)$, and $\Sigma \in \mathbb{R}^{r \times r}$ is a Bayesian weight matrix. Equation (19) is equivalent to a low-rank SVD if we define $\Sigma := \text{diag}(\sigma)$ to be *diagonal* and positive semidefinite.

To save the cost on variational parameters, we provide a weight sharing strategy to inherently Bayesianize Σ through a pre-defined projection matrix $\mathbf{P}$:

$$f_{\Delta}(\mathbf{x}) = \mathbf{B}\mathbf{Z}(\kappa(\mathbf{A}\mathbf{x}) + \mathbf{P}\mathbf{A}\mathbf{x}). \quad (20)$$

Therefore, $\mu(\mathbf{A}\mathbf{x})$ can be interpreted as a *residual* connection incorporated into the low-rank non-linearity $g(\mathbf{A}\mathbf{x})$.

4.3. Amortized Network and VI

Given the low-rank GP adaptation in (20), we perform variational inference (VI) and consider a variational family of mean-field Gaussian posteriors on random weights $q(\mathbf{Z}) = \mathcal{N}(\mathbf{Z} \mid \mathbf{M}_{\mathbf{Z}}, \Omega_{\mathbf{Z}}^2)$, parameterized by $\theta := \{\mathbf{M}_{\mathbf{Z}}, \Omega_{\mathbf{Z}}^2\}$.

As discussed in Section 2.2, amortized VI is designed to approximate posterior distributions using another input-dependent NN $f_{\phi}^{\text{inf}}(\cdot)$. Given a low-rank feature $\mathbf{A}\mathbf{x}$, we use an amortization network to output variational parameters $\theta = f_{\phi}^{\text{inf}}(\mathbf{A}\mathbf{x})$. Therefore, the amortized variational posterior is $q_{\phi}(\mathbf{Z} \mid \mathbf{x})$. The training objective is given by the VFE loss

$$\mathcal{L}(\mathbf{A}, \mathbf{B}, \phi) = -\mathbb{E}_{q_{\phi}}[\log P(\mathcal{D} \mid \mathbf{A}, \mathbf{B}, \theta)] + \text{KL}[q_{\phi}(\theta \mid \mathbf{x}) \parallel p_{\phi}(\theta)]. \quad (21)$$

During inference, we predict a new $\mathbf{x}^*$ by drawing S samples from the variational distribution:

$$P(\mathbf{y}^* \mid \mathbf{x}^*, \mathcal{D}) \approx \frac{1}{S} \sum_{i=1}^S p(\mathbf{y}^* \mid \mathbf{x}^*, \mathbf{Z}_i), \quad (22)$$

where $\mathbf{Z}_i \sim q_{\phi}(\mathbf{Z} \mid \mathbf{x}_i)$ for $i = 1, \dots, S$.

The amortized network $f_{\phi}^{\text{inf}}(\cdot)$ shares the same neural feature extractor used to learn the latent feature $\mathbf{x}$, and therefore the

input projection neural network becomes part of variational inference that helps to mitigate overfitting. Details can be found in Appendix A.5.

Complexity. Although Bayesian inference in GPan-LoRA is confined to a r^2 -dimensional adaptation space, the computational complexity additionally depends on the number of inducing points l used to approximate the underlying GP in (16), reflecting the standard trade-off in sparse GP approximations. In contrast, BLoB assigns Bayesian priors directly to LoRA factors, leading to a number of Bayesian parameters that grow with the full input dimension, i.e. $O(nr)$ where $n \gg lr$ in usual. C-LoRA similarly performs Bayesian inference in the r^2 -dimensional space; however, it does not incorporate a function-space GP prior and therefore lacks the rigorous UQ guarantees provided by GPs.

Figure 2(c) shows the modular architecture of our GPan-LoRA, which requires only $O(lr^2)$ Bayesian variables through a low-rank GPan, making the computation, memory, and number of parameters comparable to the state-of-the-art Bayesian LoRA methods.

5. Experiments

In this section, we compare the proposed GPan-LoRA with existing LLM fine-tuning methods with UQ, and answer the following key research questions:

- How effective is GPan-LoRA in quantifying uncertainty, especially under distribution shifts? (Sec. 5.2 and Sec. 5.3)
- How do different components of GPan-LoRA affect the performance of UQ? (Sec. 5.4)
- Is GPan-LoRA aware of distribution shift? (Sec. 5.5)

5.1. Experimental Settings

Base Models and Datasets. To comprehensively demonstrate the UQ capability of our model in language understanding, we conduct experiments on two pretrained base models that are commonly used in the LLMs finetuning community: LLaMA3.1-8B (Grattafiori et al., 2024) and Qwen3-8B (Yang et al., 2025), and eight publicly available datasets: Winogrande-Small, Winogrande-Medium (Sakaguchi et al., 2021), ARC-Challenge, ARC-Easy (Clark et al., 2018), OBQA (Mihaylov et al., 2018), BoolQ (Clark et al., 2019), MMLU-Chem, and MMLU-Phy (Hendrycks et al., 2020). We focus our discussion based on the results on LLaMA3.1-8B as additional results on Qwen3-8B in Appendix C.4 show similar trends.

Baselines. We compare against several recent UQ methods with LoRA, including Monte-Carlo Dropout (MCD, Gal & Ghahramani, 2016), Deep Ensemble (DeepEns, Wang et al., 2023), Laplace-LoRA (LAP, Yang et al., 2023), Bayesian

LoRA by Backprop (BLoB, Wang et al., 2024), and Contextual LoRA (C-LoRA, Rahmati et al., 2025). We also include deterministic LoRA (MLE, Hu et al., 2022) for a comprehensive comparison.

Fine-tuning Setups. To ensure a fair and robust comparison, we partitioned the original training set, reserving 20% as a held-out validation set. We capped the training at 5,000 steps, ran validation every 500 steps, and used the checkpoint with the highest validation accuracy for the final evaluation. Detailed hyperparameter configurations are provided in Appendix B.2.

Metrics. The datasets consist of either binary or multiple-choice QA problems. We adopt three commonly used metrics for UQ evaluation (Wang et al., 2024; Rahmati et al., 2025) on these datasets: accuracy (ACC), expected calibrated error (ECE), and negative log-likelihood (NLL).

5.2. Performance Comparison

In-Distribution. We first evaluate GPan-LoRA on in-distribution (ID) datasets as reported in Table 2. Overall, GPan-LoRA achieves competitive accuracy while providing strong uncertainty estimates.

Predictive Accuracy. On ID tasks, GPan-LoRA maintains comparable accuracy to deterministic fine-tuning (MLE) and other UQ baselines, indicating that our Bayesianization does not compromise task adaptation. While no single method uniformly dominates across all datasets, GPan-LoRA remains within a competitive range and preserves the base model’s generative capability.

Uncertainty Calibration. In terms of calibration, GPan-LoRA is consistently strong and achieves the best or second-best ECE across all datasets, outperforming recent Bayesian methods on these benchmarks. For NLL, although LAP and C-LoRA are highly competitive, GPan-LoRA attains the lowest NLL on ARC-C/E and OBQA, and is comparable to the best method on other datasets. These results suggest that GPan-LoRA can match strong Bayesian baselines on ID performance while improving calibration on multiple tasks.

Out-of-Distribution. We further assess robustness under distribution shift, where models trained on OBQA are evaluated directly on unseen targets. We group targets into *small* and *large* shifts as illustrated in the right part of Table 2.

Small Distribution Shifts. On mild shifts (ARC-C/E), although GPan-LoRA shows lower ACC compared to LAP, it does not compromise calibration quality. In fact, GPan-LoRA remains a top-tier performer in uncertainty quantification: it achieves the best and second-best ECE on ARC-E and ARC-C, respectively. This indicates that while GPan-LoRA may be less aggressive in fitting the source domain’s

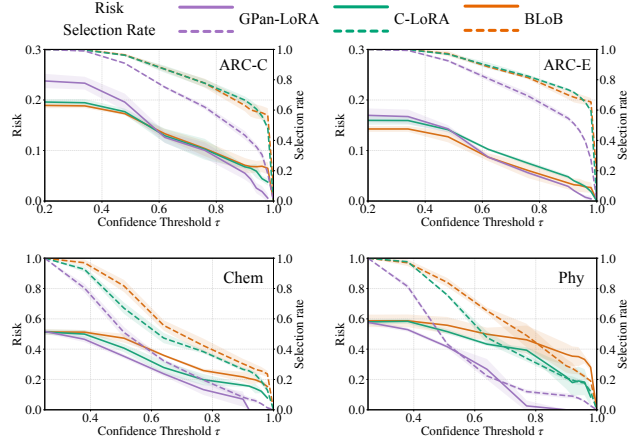


Figure 3. Risk and selection rate vs. confidence threshold τ under distribution shift. Solid: risk; dashed: selection rate; shading: ± 1 std over seeds.

accuracy peaks, it successfully preserves reliable probabilistic estimates during near-domain transfer. The illustrative example in Figure 1 also confirms this: GPan-LoRA may have a more conservative prediction error but better uncertainty calibration. Further discussion can be found in Section 5.3 and Appendix B.1.

Large Distribution Shifts. Under severe shifts to specialized STEM domains (Chem/Phy), the robustness of GPan-LoRA becomes the decisive factor. It achieves *state-of-the-art* UQ quality, recording the lowest ECE on both Chem and Phy (11.31/13.29) and the lowest NLL (1.19/1.25). Crucially, it outperforms the strongest baselines by a large margin: compared to LAP (ECE 14.59/14.00) and BLoB (ECE 24.45/32.16), GPan-LoRA reduces calibration error significantly. We hypothesize that this consistent superiority stems from GPan-LoRA’s ability to better capture epistemic uncertainty when the input distribution deviates substantially from the training domain. Detailed discussion can be found in Section 5.5.

Last Layer Analysis. GPan-LoRA is demonstrated achieving more robust performances under distribution shift with comparable computation, memory, and number of parameters to the other state-of-the-art approaches. For budget-constrained application scenarios, it may be desirable to adopt a resource-light option without sacrificing too much performance. Motivated by previous work suggesting that incorporating uncertainty only in the last layer can preserve most of the advantages of full Bayesian treatment while significantly reducing computational cost (Wilson et al., 2016a; Harrison et al., 2024), we have investigated a lightweight variant of GPan that Bayesianizes only the last LoRA layer, denoted as **LL GPan-LoRA**. We evaluate its UQ performance under the same experimental setup as the full GPan-LoRA model, with results summarized in Table 2. As expected, restricting Bayesianization to the

Table 2. Performance of different uncertainty estimation methods applied to LoRA on LLaMA3.1-8B pre-trained weights. Accuracy (ACC) and Expected Calibration Error (ECE) are reported in percentages, where “↑” and “↓” indicate that higher and lower values are preferred, respectively. The left six columns correspond to the in-distribution setting, while the right four columns report performance under distribution shift. Best and second-best ECE and NLL results are highlighted in **boldface** and underline, respectively.

Metric	Method	In-Distribution Datasets						Out-of-Distribution Datasets (OBQA→X)			
								Small Dist. Shift		Large Dist. Shift	
		WG-S	ARC-C	ARC-E	WG-M	OBQA	BoolQ	ARC-C	ARC-E	Chem	Phy
ACC (↑)	MLE	76.18±1.45	79.39±0.67	91.43±1.27	81.27±1.70	86.26±0.41	89.60±0.80	80.40±1.35	85.15±0.90	48.33±1.52	39.33±2.88
	MCD	75.50±0.71	80.85±1.27	90.25±0.36	81.90±0.53	86.66±1.33	89.43±0.67	80.51±0.51	85.91±0.63	48.00±2.64	42.00±4.00
	DeepEns	76.74±1.07	80.91±0.71	91.66±0.26	81.85±1.49	87.00±0.20	89.88±0.24	81.30±0.39	85.97±0.44	44.00±3.46	45.66±1.52
	LAP	74.55±0.51	81.47±0.42	91.48±0.36	80.40±0.27	85.73±0.34	87.87±0.36	80.68±1.02	88.01±0.71	47.00±2.00	44.44±0.57
	BLoB	72.12±0.43	79.73±1.75	90.84±0.63	79.40±0.31	86.86±0.41	89.76±0.16	81.64±1.08	86.67±1.78	49.00±1.00	43.00±4.58
	C-LoRA	73.76±2.28	80.40±0.67	89.61±0.46	76.26±2.06	87.20±0.34	89.54±0.24	80.74±0.67	83.68±0.44	49.00±1.73	42.66±2.08
	GPan-LoRA	72.41±0.24	80.06±1.01	91.84±0.61	79.82±0.44	86.66±0.94	87.20±3.29	75.78±1.36	82.74±1.23	48.00±2.00	42.33±0.57
	LL GPan-LoRA	75.60±0.09	79.27±2.24	90.72±0.86	78.63±0.55	86.13±0.83	88.97±0.30	79.84±1.52	84.03±0.56	49.66±2.08	39.33±4.50
ECE (↓)	MLE	22.43±0.94	17.01±1.80	6.33±2.21	13.43±2.62	6.93±2.84	4.24±1.73	16.16±1.34	11.30±0.34	32.60±2.47	41.61±1.17
	MCD	23.36±0.77	15.42±3.50	4.55±2.60	15.24±1.52	9.01±0.59	5.51±1.17	11.74±2.81	8.48±1.28	24.33±5.09	32.07±6.98
	DeepEns	21.80±1.12	17.50±1.07	6.93±0.64	14.07±0.24	9.98±0.65	6.05±0.17	9.02±0.09	9.69±1.39	25.37±3.32	28.45±1.46
	LAP	7.78±1.92	5.81±0.96	<u>3.50±0.65</u>	4.31±0.26	<u>3.67±0.21</u>	<u>1.75±0.07</u>	5.01±0.93	<u>3.43±1.11</u>	14.59±2.41	14.00±1.87
	BLoB	17.29±0.59	8.68±4.33	4.36±2.45	13.66±1.75	5.78±0.61	7.28±0.37	8.26±0.94	5.37±1.78	24.45±1.20	32.16±4.57
	C-LoRA	14.50±1.43	5.68±2.06	4.16±0.84	9.91±1.67	5.35±0.34	4.35±0.16	8.16±1.74	7.20±2.22	19.39±2.22	25.68±2.98
	GPan-LoRA	<u>13.39±2.67</u>	4.60±0.88	2.36±0.85	<u>6.85±1.44</u>	3.12±0.64	1.45±0.62	<u>6.46±0.87</u>	3.31±0.37	<u>11.31±2.75</u>	13.29±1.20
	LL GPan-LoRA	15.72±3.18	7.93±2.27	3.87±0.54	8.25±2.49	6.07±1.70	2.13±0.20	8.01±0.56	3.67±0.66	14.37±2.24	13.67±3.00
NLL (↓)	MLE	1.92±0.18	1.35±0.46	0.43±0.12	0.63±0.10	0.41±0.03	0.28±0.02	1.10±0.03	0.78±0.05	1.96±0.11	2.38±0.17
	MCD	2.28±0.13	0.85±0.12	0.35±0.12	0.92±0.23	0.50±0.06	0.30±0.02	0.74±0.13	0.45±0.09	1.47±0.20	1.72±0.32
	DeepEns	1.78±0.15	1.51±0.03	0.48±0.14	0.72±0.12	0.57±0.10	0.31±0.01	0.59±0.01	0.48±0.01	1.26±0.04	1.36±0.02
	LAP	0.56±0.02	0.51±0.01	0.26±0.01	0.45±0.01	<u>0.38±0.01</u>	<u>0.27±0.01</u>	0.50±0.02	0.35±0.02	1.28±0.03	<u>1.29±0.01</u>
	BLoB	0.94±0.01	0.65±0.04	0.36±0.07	0.72±0.03	0.46±0.04	0.41±0.01	0.68±0.27	0.44±0.05	1.61±0.07	1.91±0.10
	C-LoRA	0.70±0.06	0.56±0.01	0.29±0.01	0.55±0.03	0.38±0.01	0.27±0.01	0.60±0.04	0.44±0.01	1.35±0.06	1.51±0.06
	GPan-LoRA	<u>0.69±0.08</u>	0.51±0.02	0.24±0.01	<u>0.48±0.01</u>	0.37±0.02	0.30±0.06	0.60±0.03	0.46±0.01	1.19±0.01	1.25±0.01
	LL GPan-LoRA	0.78±0.14	0.58±0.05	0.28±0.01	0.51±0.03	0.39±0.01	0.26±0.01	<u>0.56±0.03</u>	<u>0.38±0.02</u>	<u>1.25±0.05</u>	1.25±0.06

last layer leads to moderate performance degradation across most datasets. Nevertheless, this efficient variant remains competitive with existing baselines and notably achieves strong performance under large distribution shifts, where it consistently outperforms prior methods. These results suggest that last-layer Bayesianization offers a favorable trade-off between computational efficiency and robust uncertainty estimation. Detailed efficiency analysis can be found in Appendix C.1.

5.3. Selective Prediction Under Distribution Shift

In this section, we study selective prediction under distribution shift by allowing the model to abstain when its confidence is below a threshold τ . We focus on end-to-end Bayesian LoRA methods that are trained and deployed under comparable computational budgets. For each input x , the model outputs a predictive distribution over C answer options. We define confidence as $\max_{c \in [C]} p_{\theta}(y = c | x)$. The model selects an example if its confidence exceeds τ , yielding the **selection rate**: $\text{Sel}(\tau) = \mathbb{P}(\max_c p_{\theta}(y = c | x) \geq \tau)$, and the corresponding **risk** on the selected subset: $\text{Risk}(\tau) = \mathbb{E}[\mathbb{I}(\hat{y}(x) \neq y) | \max_c p_{\theta}(y = c | x) \geq \tau]$. We sweep τ from $1/C$ to 1 and report both $\text{Risk}(\tau)$ and $\text{Sel}(\tau)$, as shown in Figure 3.

Small Distribution shift: reliability emerges at high con-

fidence. Under the small distribution shift regime (Table 2), GPan-LoRA does not yield the best overall accuracy. Nonetheless, Figures 3(a-b) show that as τ increases, GPan-LoRA’s risk decreases much faster than the baselines, and in the high-confidence regime it consistently achieves the lowest risk. This indicates that even when average accuracy is not dominant, GPan-LoRA better identifies instances where it is likely to be correct and abstains on the rest, making it preferable when the application requires *extremely reliable* predictions.

Large Distribution shift: lower risk without over-confidence. Under a large distribution shift as shown in Figures 3(c-d), GPan-LoRA’s risk is consistently below both baselines across a wide range of τ . Importantly, when viewed together with the selection-rate curves, GPan-LoRA does not maintain an artificially high selection rate at the expense of errors; instead, it reduces the selection rate appropriately as τ increases. This behavior suggests that GPan-LoRA is less over-confident under severe shift: it assigns lower confidence to unfamiliar or hard examples, abstains more often, and consequently maintains lower selective risk than baselines. At the same time, when operating at high-confidence thresholds, GPan-LoRA achieves higher prediction accuracy, indicating that its confidence is well aligned with true predictive reliability.

Overall, these results highlight that selective prediction pro-

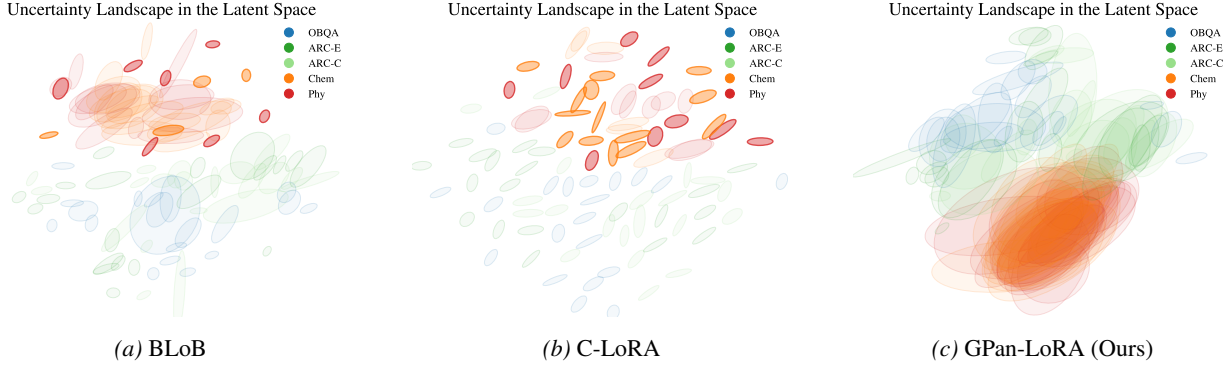


Figure 4. t-SNE visualization of the latent embeddings under distribution shift.

Table 3. Ablation: performance of GPan-LoRA and its variants under distribution shift.

Metric	Method	In-Dist.	Out-of-Dist. (OBQA→X)				
			Small Dist. Shift		Large Dist. Shift		
		OBQA	ARC-C	ARC-E	Chem	Phy	
ACC ↑	GPan-LoRA	86.66±0.94	75.78±1.36	82.74±1.23	48.00±2.00	42.33±0.51	
	w/o AN	87.27±0.83	80.96±0.19	85.79±1.23	42.00±3.00	41.66±3.21	
	w/o WS	84.73±0.80	75.78±1.18	80.16±0.61	44.33±1.52	38.66±3.21	
ECE ↓	GPan-LoRA	3.12±0.64	6.46±0.87	3.31±0.37	11.31±2.75	13.29±1.20	
	w/o AN	3.71±0.97	6.35±0.38	4.14±1.03	22.60±0.94	26.15±3.49	
	w/o WS	<u>3.70±1.00</u>	7.92±0.23	4.62±0.59	19.67±2.51	22.52±0.91	
NLL ↓	GPan-LoRA	<u>0.37±0.02</u>	<u>0.60±0.03</u>	<u>0.46±0.01</u>	<u>1.19±0.01</u>	<u>1.25±0.01</u>	
	w/o AN	0.36±0.01	0.57±0.03	0.39±0.04	1.37±0.02	1.50±0.05	
	w/o WS	0.39±0.01	0.66±0.01	0.53±0.02	1.34±0.04	1.39±0.03	

vides a complementary view to average accuracy: GPan-LoRA is particularly advantageous in high-reliability operating regions and under large distribution shift, where well-calibrated abstention is crucial.

5.4. Ablation Study

We investigate the contributions of the amortized network and weight sharing in GPan-LoRA by ablating each component individually, resulting in two variants: w/o amortized network (w/o AN) and w/o weight sharing (w/o WS), as shown in Table 3.

W/o amortized network. In this variant, we remove the amortized inference mechanism introduced in Section 4.3, the variational parameters are optimized independently without amortization. While this variant achieves comparable or slightly higher in-distribution accuracy, it consistently exhibits worse uncertainty calibration under both small and large distribution shifts. This ablation suggests that amortized inference is critical for learning input-aware variational posteriors, enabling GPan-LoRA to maintain reliable uncertainty estimates when encountering unseen inputs.

W/o weight sharing. For the w/o WS variant, the mean function and kernel function in Equation (17) no longer share parameters, leading to a larger number of independent Bayesian parameters. As shown in Table 3, this modification results in consistent performance degradation across both

accuracy and uncertainty metrics, especially under large distribution shifts. The increased ECE and NLL indicate that removing weight sharing negatively affects posterior learning, likely due to over-parameterization and reduced statistical efficiency.

5.5. Visualization of Uncertainty Landscape

To further investigate how GPan-LoRA quantifies uncertainty under varying degrees of distribution shift, we visualize the variance of latent space embeddings. Following the experimental protocol for uncertainty visualization, we extract the output embeddings from the final transformer layer for two variational inference based baselines and our proposed GPan-LoRA. Detailed visualization setup is provided in Appendix B.3.

Results and Observations. As illustrated in Figure 4, our comparative analysis yields several key insights:

Overconfidence in Baselines. The baseline models exhibit varying degrees of overconfidence when encountering OOD data. Specifically, under distribution shifts, these baselines yield tightly clustered embeddings (highlighted with bold border in the figure). This indicates a failure to capture epistemic uncertainty, as the models provide low-variance predictions for samples far from the training distribution.

Robust Uncertainty Estimation in GPan-LoRA. In contrast, GPan-LoRA demonstrates a superior ability to remain “cautious.” As shown in Figure 4(c), GPan-LoRA maintains significantly higher uncertainty (manifested as larger, more dispersed ellipses) for OOD MMLU samples compared to the ID OBQA samples. This behavior aligns with the motivating regression example in Figure 1, confirming that GPan-LoRA successfully replicates the desirable property of GPs of expanding uncertainty estimates in OOD regions within the representation space induced by LLMs.

Sensitivity to Shift Magnitude. Our model also distinguishes between different levels of shift: while the uncertainty for ARC datasets is moderately increased, the expansion is most pronounced for the MMLU datasets. Combined with

discussion in Section 5.3, this confirms that GPan-LoRA effectively leverages its Bayesian framework to mitigate the risk of overconfident mispredictions in complex, unseen domains.

6. Conclusion

In this work, we presented GPan-LoRA, a Bayesian LoRA framework via Gaussian Process amortized networks, that brings principled uncertainty quantification to parameter-efficient fine-tuning of LLMs. By reformulating sparse GPs as low-rank Bayesian adapters and learning their variational posteriors through amortized networks, GPan-LoRA achieves strong uncertainty quality while operating entirely in a compact r^2 -dimensional adaptation space. Experiments demonstrate that this design yields well-calibrated uncertainty and mitigates overconfidence under distribution shift without sacrificing efficiency, positioning GPan-LoRA as a practical and theoretically grounded solution for uncertainty-aware LLM adaptation.

Acknowledgements

This work was supported in part by the U.S. National Science Foundation (NSF) grants SHF-2215573, and IIS-2212419. The work of W. Zhao and C. Tian was supported in part by the NSF via grants DMS-2312173 and ECCS-2433631. Portions of this research were conducted with the advanced computing resources provided by Texas A&M High Performance Research Computing.

Impact Statement

This paper presents work whose goal is to advance the fields of Gaussian Processes, Bayesian Learning, and Large Language Models. There are many potential societal consequences of our work, especially in tackling challenges in achieving trustworthy AI/ML.

References

- Abdar, M., Pourpanah, F., Hussain, S., Rezazadegan, D., Liu, L., Ghavamzadeh, M., Fieguth, P., Cao, X., Khosravi, A., Acharya, U. R., et al. A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information fusion*, 76:243–297, 2021.
- Balabanov, O. and Linander, H. Uncertainty quantification in fine-tuned llms using lora ensembles. *arXiv preprint arXiv:2402.12264*, 2024.
- Boluki, S., Ardywibowo, R., Dadaneh, S. Z., Zhou, M., and Qian, X. Learnable Bernoulli dropout for Bayesian deep learning. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics*, 2020.
- Bradshaw, J., Matthews, A. G. d. G., and Ghahramani, Z. Adversarial examples, uncertainty, and transfer testing robustness in gaussian process hybrid deep networks. *arXiv preprint arXiv:1707.02476*, 2017.
- Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., et al. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 15(3):1–45, 2024.
- Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*, 2019.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Damianou, A. and Lawrence, N. D. Deep gaussian processes. In *Artificial intelligence and statistics*, pp. 207–215. PMLR, 2013.
- de Souza, D. A., Nikitin, A., John, S., Ross, M., Álvarez, M. A., Deisenroth, M., Gomes, J. P., Mesquita, D., and Mattos, C. L. Thin and deep gaussian processes. *Advances in Neural Information Processing Systems*, 36: 13138–13148, 2023.
- Fan, X., Zhang, S., Tanwisuth, K., Qian, X., and Zhou, M. Contextual dropout: an efficient sample-dependent dropout. In *Proceedings of the 9th International Conference on Learning Representations*, ICLR’21, 2021.
- Gal, Y. and Ghahramani, Z. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pp. 1050–1059. PMLR, 2016.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Harrison, J., Willes, J., and Snoek, J. Variational bayesian last layers. *arXiv preprint arXiv:2404.11599*, 2024.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- Hensman, J., Fusi, N., and Lawrence, N. D. Gaussian processes for big data. *arXiv preprint arXiv:1309.6835*, 2013.

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Jacot, A., Gabriel, F., and Hongler, C. Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems*, 2018.
- Jolliffe, I. T. and Cadima, J. Principal component analysis: a review and recent developments. *Philosophical transactions of the royal society A: Mathematical, Physical and Engineering Sciences*, 374(2065):20150202, 2016.
- Kaddour, J., Harris, J., Mozes, M., Bradley, H., Raileanu, R., and McHardy, R. Challenges and applications of large language models. *arXiv preprint arXiv:2307.10169*, 2023.
- Kasneci, E., Seßler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günnemann, S., Hüllermeier, E., et al. Chatgpt for good? on opportunities and challenges of large language models for education. *Learning and individual differences*, 103: 102274, 2023.
- Lakshminarayanan, B., Pritzel, A., and Blundell, C. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30, 2017.
- Maaten, L. v. d. and Hinton, G. Visualizing data using t-sne. *Journal of machine learning research*, 9(Nov): 2579–2605, 2008.
- Majumder, S., Dong, L., Doudi, F., Cai, Y., Tian, C., Kalathil, D., Ding, K., Thatte, A. A., Li, N., and Xie, L. Exploring the capabilities and limitations of large language models in the electric energy sector. *Joule*, 8(6): 1544–1549, 2024.
- Matias, A. L., Mattos, C. L., Gomes, J. P. P., and Mesquita, D. Amortized variational deep kernel learning. In *Forty-first International Conference on Machine Learning*, 2024.
- Mihaylov, T., Clark, P., Khot, T., and Sabharwal, A. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789*, 2018.
- Neal, R. M. Bayesian leaning for neural networks, 1996.
- Ober, S. W., Rasmussen, C. E., and van der Wilk, M. The promises and pitfalls of deep kernel learning. In *Uncertainty in Artificial Intelligence*, pp. 1206–1216. PMLR, 2021.
- Onal, E., Flöge, K., Caldwell, E., Sheverdin, A., and Fortuin, V. Gaussian stochastic weight averaging for bayesian low-rank adaptation of large language models. *arXiv preprint arXiv:2405.03425*, 2024.
- Rahmati, A. H., Jantre, S., Zhang, W., Wang, Y., Yoon, B.-J., Urban, N. M., and Qian, X. C-LoRA: Contextual low-rank adaptation for uncertainty estimation in large language models. *arXiv preprint arXiv:2505.17773*, 2025.
- Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Salimbeni, H. and Deisenroth, M. Doubly stochastic variational inference for deep gaussian processes. *Advances in neural information processing systems*, 30, 2017.
- Shi, H., Wang, Y., Han, L., Zhang, H., and Wang, H. Training-free bayesianization for low-rank adapters of large language models. *arXiv preprint arXiv:2412.05723*, 2024.
- Teubner, T., Flath, C. M., Weinhardt, C., Van Der Aalst, W., and Hinz, O. Welcome to the era of chatgpt et al. *Business & information systems engineering*, 65(2):95–101, 2023.
- Thirunavukarasu, A. J., Ting, D. S. J., Elangovan, K., Gutierrez, L., Tan, T. F., and Ting, D. S. W. Large language models in medicine. *Nature medicine*, 29(8):1930–1940, 2023.
- Titsias, M. Variational learning of inducing variables in sparse gaussian processes. In *Artificial intelligence and statistics*, pp. 567–574. PMLR, 2009.
- Wang, X., Aitchison, L., and Rudolph, M. LoRA ensembles for large language model fine-tuning. *arXiv preprint arXiv:2310.00035*, 2023.
- Wang, Y., Shi, H., Han, L., Metaxas, D., and Wang, H. Blob: Bayesian low-rank adaptation by backpropagation for large language models. *arXiv preprint arXiv:2406.11675*, 2024.
- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.
- Williams, C. Computing with infinite networks. *Advances in neural information processing systems*, 9, 1996.
- Williams, C. K. and Rasmussen, C. E. *Gaussian processes for machine learning*, volume 2. MIT press Cambridge, MA, 2006.

- Wilson, A. G., Hu, Z., Salakhutdinov, R., and Xing, E. P. Deep kernel learning. In *Artificial intelligence and statistics*, pp. 370–378. PMLR, 2016a.
- Wilson, A. G., Hu, Z., Salakhutdinov, R. R., and Xing, E. P. Stochastic variational deep kernel learning. *Advances in neural information processing systems*, 29, 2016b.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yang, A. X., Robeyns, M., Wang, X., and Aitchison, L. Bayesian low-rank adaptation for large language models. *arXiv preprint arXiv:2308.13111*, 2023.
- Zhao, W., Chen, H., Liu, T., Tuo, R., and Tian, C. From deep additive kernel learning to last-layer bayesian neural networks via induced prior approximation. *arXiv preprint arXiv:2502.10540*, 2025.
- Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

A. Background: Gaussian Process and Variational Inference

A.1. Exact Gaussian Processes

A Gaussian Process (GP) defines a distribution over real-valued functions $f : \mathbb{R}^D \rightarrow \mathbb{R}$ such that any finite collection of function values follows a joint Gaussian distribution. Formally,

$$f(\cdot) \sim \mathcal{GP}(m(\cdot), k(\cdot, \cdot)), \quad (23)$$

where $m(\cdot)$ is the mean function (typically set to zero) and $k(\cdot, \cdot)$ is a positive-definite kernel.

Given training inputs $\mathbf{X} = \{x_i\}_{i=1}^N$ and noisy observations

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2), \quad (24)$$

the prior over latent function values $\mathbf{f} = f(\mathbf{X})$ is

$$p(\mathbf{f}) = \mathcal{N}(\mathbf{0}, \mathbf{K}_{\mathbf{XX}}), \quad (25)$$

where $(\mathbf{K}_{\mathbf{XX}})_{ij} = k(x_i, x_j)$. The marginal likelihood is

$$p(\mathbf{y}) = \mathcal{N}(\mathbf{0}, \mathbf{K}_{\mathbf{XX}} + \sigma^2 \mathbf{I}). \quad (26)$$

For a test input x_* , let $f_* = f(x_*)$. The joint prior over $(\mathbf{f}, f_*)$ is

$$\begin{bmatrix} \mathbf{f} \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K}_{\mathbf{XX}} & \mathbf{k}_{\mathbf{X}*} \\ \mathbf{k}_{*\mathbf{X}} & k_{**} \end{bmatrix}\right), \quad (27)$$

where $\mathbf{k}_{\mathbf{X}*} = k(\mathbf{X}, x_*)$. Conditioning on observations $\mathbf{y}$ yields the exact GP posterior:

$$p(f_* | \mathbf{y}) = \mathcal{N}(\mu_*, \sigma_*^2), \quad (28)$$

$$\mu_* = \mathbf{k}_{*\mathbf{X}}(\mathbf{K}_{\mathbf{XX}} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}, \quad (29)$$

$$\sigma_*^2 = k_{**} - \mathbf{k}_{*\mathbf{X}}(\mathbf{K}_{\mathbf{XX}} + \sigma^2 \mathbf{I})^{-1} \mathbf{k}_{\mathbf{X}*}. \quad (30)$$

While exact GPs provide closed-form posterior inference and well-calibrated uncertainty estimates, their computational complexity scales as $\mathcal{O}(N^3)$ in training and $\mathcal{O}(N^2)$ in prediction, which limits applicability to large-scale datasets.

A.2. Sparse Variational Gaussian Processes with Inducing Points

Sparse variational GP (SVGP) methods alleviate the cubic complexity by introducing a set of $M \ll N$ inducing points $\mathbf{Z} = \{z_m\}_{m=1}^M$ with corresponding inducing variables $\mathbf{u} = f(\mathbf{Z})$. Under the GP prior,

$$p(\mathbf{u}) = \mathcal{N}(\mathbf{0}, \mathbf{K}_{\mathbf{ZZ}}), \quad p(\mathbf{f} | \mathbf{u}) = \mathcal{N}(\mathbf{K}_{\mathbf{XZ}} \mathbf{K}_{\mathbf{ZZ}}^{-1} \mathbf{u}, \mathbf{K}_{\mathbf{XX}} - \mathbf{Q}_{\mathbf{XX}}), \quad (31)$$

where $\mathbf{Q}_{\mathbf{XX}} = \mathbf{K}_{\mathbf{XZ}} \mathbf{K}_{\mathbf{ZZ}}^{-1} \mathbf{K}_{\mathbf{ZX}}$.

Variational inference is applied by introducing a tractable variational distribution over inducing variables,

$$q(\mathbf{u}) = \mathcal{N}(\mathbf{m}, \mathbf{S}), \quad (32)$$

which induces an approximate posterior over function values

$$q(\mathbf{f}) = \int p(\mathbf{f} | \mathbf{u}) q(\mathbf{u}) d\mathbf{u}. \quad (33)$$

The variational parameters $(\mathbf{m}, \mathbf{S})$ are optimized by maximizing the evidence lower bound (ELBO):

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q(\mathbf{f})} [\log p(\mathbf{y} | \mathbf{f})] - \text{KL}(q(\mathbf{u}) \| p(\mathbf{u})). \quad (34)$$

Sparse variational GPs reduce training complexity to $\mathcal{O}(NM^2)$ and prediction to $\mathcal{O}(M^2)$, while retaining a principled Bayesian treatment of uncertainty.

A.3. From sparse GP approximations to variational Bayesian networks

Although SVGP reduces the complexity from $O(N^3)$ to $O(NM^2)$, it still misaligns the training and inference techniques in modern deep neural networks. To further improve the computation, we provide a functional interpretation of sparse GP approximations that reveals their close connection to variational Bayesian neural networks (VBNs) with structured, kernel-induced features.

Consider a latent function $g(\cdot)$ endowed with a GP prior. In sparse variational GPs, a set of inducing inputs $\mathbf{u} \in \mathbb{R}^M$ is introduced, together with inducing variables $g(\mathbf{u})$. Conditioned on $g(\mathbf{u})$, the GP prior admits the approximation

$$g(\mathbf{x}) \approx k(\mathbf{x}, \mathbf{u}) k(\mathbf{u}, \mathbf{u})^{-1} g(\mathbf{u}), \quad (35)$$

which corresponds to projecting the function onto the subspace spanned by kernel evaluations at inducing points. This approximation becomes exact as $M \rightarrow \infty$ under mild conditions.

We now specialize to the setting where the GP is defined over a low-rank linear projection of the input, i.e.

$$g(\mathbf{x}) = g(\mathbf{A}\mathbf{x}),$$

with $\mathbf{A} = [\mathbf{a}_1, \dots, \mathbf{a}_r]^\top \in \mathbb{R}^{r \times n}$. Assuming an additive kernel over projected coordinates, the kernel admits the decomposition

$$k(\mathbf{A}\mathbf{x}, \mathbf{u}) = \sum_{i=1}^r k(\mathbf{a}_i^\top \mathbf{x}, \mathbf{u}). \quad (36)$$

Applying the approximation in (35) yields

$$g(\mathbf{A}\mathbf{x}) \approx k(\mathbf{A}\mathbf{x}, \mathbf{u}) k(\mathbf{u}, \mathbf{u})^{-1} g(\mathbf{u}) \quad (37)$$

$$= \sum_{i=1}^r k(\mathbf{a}_i^\top \mathbf{x}, \mathbf{u}) k(\mathbf{u}, \mathbf{u})^{-1} g(\mathbf{u}). \quad (38)$$

Let $k(\mathbf{u}, \mathbf{u}) = \mathbf{L}_u \mathbf{L}_u^\top$ denote the Cholesky factorization of the inducing kernel matrix. Then

$$g(\mathbf{A}\mathbf{x}) \approx \sum_{i=1}^r k(\mathbf{a}_i^\top \mathbf{x}, \mathbf{u}) [\mathbf{L}_u^\top]^{-1} \mathbf{L}_u^{-1} g(\mathbf{u}) \quad (39)$$

$$\triangleq \sum_{i=1}^r \mathbf{z}_i^\top \kappa(\mathbf{a}_i^\top \mathbf{x}, \mathbf{u}), \quad (40)$$

where we define

$$\kappa(\mathbf{a}_i^\top \mathbf{x}, \mathbf{u}) := k(\mathbf{a}_i^\top \mathbf{x}, \mathbf{u}) [\mathbf{L}_u^\top]^{-1}, \quad \mathbf{z}_i := \mathbf{L}_u^{-1} g(\mathbf{u}).$$

Under the GP prior, $g(\mathbf{u}) \sim \mathcal{N}(\mathbf{0}, k(\mathbf{u}, \mathbf{u}))$, which implies

$$\mathbf{z}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

and hence each $\mathbf{z}_i$ can be interpreted as a standard Gaussian latent weight vector. Consequently, the sparse GP approximation induces a Bayesian linear model in the kernel feature space $\kappa(\cdot)$, with structured weight sharing across the r projected dimensions.

This establishes an explicit equivalence between sparse GP approximations and a class of VBNS, where uncertainty is encoded in a low-dimensional latent weight space while nonlinear expressiveness is provided by kernel-induced features.

A.4. Proof of Theorem 4.1: exact GP recasting through linear transformations

Theorem: The following statements are equivalent:

- 1) There exists a zero-mean GP $\tilde{g}(\cdot)$ on $\mathbb{R}^r$ such that $g(\mathbf{x}) \stackrel{d}{=} \tilde{g}(\mathbf{A}\mathbf{x})$.
- 2) There exists a covariance function $\tilde{k}(\cdot, \cdot) : \mathbb{R}^r \times \mathbb{R}^r \rightarrow \mathbb{R}$ such that $k(\mathbf{x}, \mathbf{x}') = \tilde{k}(\mathbf{A}\mathbf{x}, \mathbf{A}\mathbf{x}')$.

Moreover, when these conditions hold, $\tilde{k}$ is positive semi-definite on the column space $\text{range}(\mathbf{A}) \equiv \{\mathbf{A}\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^r$ and therefore defines a valid GP.

Proof. We prove the equivalence of (1) and (2) by matching finite-dimensional distributions, and then show the stated positive semi-definiteness claim on $\text{range}(\mathbf{A})$.

(1) $\Rightarrow$ (2). Assume there exists a zero-mean GP $\tilde{g}$ on $\mathbb{R}^r$ such that

$$g(\mathbf{x}) \stackrel{d}{=} \tilde{g}(\mathbf{Ax}) \quad (\text{as stochastic processes indexed by } \mathbf{x} \in \mathbb{R}^n).$$

By definition of equality in distribution for stochastic processes, for any $m \in \mathbb{N}$ and any inputs $\mathbf{x}_1, \dots, \mathbf{x}_m \in \mathbb{R}^n$, the random vectors

$$(g(\mathbf{x}_1), \dots, g(\mathbf{x}_m)) \quad \text{and} \quad (\tilde{g}(\mathbf{Ax}_1), \dots, \tilde{g}(\mathbf{Ax}_m))$$

have the same distribution. Both are centered Gaussian vectors. Hence, their covariance matrices coincide entrywise:

$$\text{Cov}(g(\mathbf{x}_i), g(\mathbf{x}_j)) = \text{Cov}(\tilde{g}(\mathbf{Ax}_i), \tilde{g}(\mathbf{Ax}_j)), \quad \forall i, j \in [m].$$

Let $\tilde{k} : \mathbb{R}^r \times \mathbb{R}^r \rightarrow \mathbb{R}$ denote the covariance function of $\tilde{g}$: $\tilde{k}(\mathbf{u}, \mathbf{v}) = \text{Cov}(\tilde{g}(\mathbf{u}), \tilde{g}(\mathbf{v}))$. Then for all $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^n$,

$$k(\mathbf{x}, \mathbf{x}') := \text{Cov}(g(\mathbf{x}), g(\mathbf{x}')) = \text{Cov}(\tilde{g}(\mathbf{Ax}), \tilde{g}(\mathbf{Ax}')) = \tilde{k}(\mathbf{Ax}, \mathbf{Ax}').$$

This is exactly statement (2).

(2) $\Rightarrow$ (1). Assume there exists a function $\tilde{k} : \mathbb{R}^r \times \mathbb{R}^r \rightarrow \mathbb{R}$ such that

$$k(\mathbf{x}, \mathbf{x}') = \tilde{k}(\mathbf{Ax}, \mathbf{Ax}') \quad \forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^n. \quad (41)$$

We first show that $\tilde{k}$ is positive semi-definite on the index set $\text{range}(\mathbf{A})$, and hence defines a valid GP on that set.

Take any $m \in \mathbb{N}$, any points $\mathbf{u}_1, \dots, \mathbf{u}_m \in \text{range}(\mathbf{A})$, and any coefficients $c_1, \dots, c_m \in \mathbb{R}$. By definition of $\text{range}(\mathbf{A})$, for each i there exists $\mathbf{x}_i \in \mathbb{R}^n$ such that $\mathbf{u}_i = \mathbf{Ax}_i$. Then, using (41),

$$\sum_{i=1}^m \sum_{j=1}^m c_i c_j \tilde{k}(\mathbf{u}_i, \mathbf{u}_j) = \sum_{i=1}^m \sum_{j=1}^m c_i c_j \tilde{k}(\mathbf{Ax}_i, \mathbf{Ax}_j) = \sum_{i=1}^m \sum_{j=1}^m c_i c_j k(\mathbf{x}_i, \mathbf{x}_j).$$

Since k is a covariance function of the GP g , it is positive semi-definite on $\mathbb{R}^n$, so the right-hand side is ≥ 0 . Therefore,

$$\sum_{i,j} c_i c_j \tilde{k}(\mathbf{u}_i, \mathbf{u}_j) \geq 0 \quad \text{for all choices of } \{\mathbf{u}_i\} \subset \text{range}(\mathbf{A}), \{c_i\} \subset \mathbb{R}.$$

Thus $\tilde{k}$ is positive semi-definite on $\text{range}(\mathbf{A})$.

By the standard existence theorem for Gaussian processes (e.g., via Kolmogorov extension), there exists a centered GP

$$\tilde{g} : \text{range}(\mathbf{A}) \rightarrow \mathbb{R}$$

with covariance function $\tilde{k}$ restricted to $\text{range}(\mathbf{A}) \times \text{range}(\mathbf{A})$, i.e. $\text{Cov}(\tilde{g}(\mathbf{u}), \tilde{g}(\mathbf{v})) = \tilde{k}(\mathbf{u}, \mathbf{v})$ for all $\mathbf{u}, \mathbf{v} \in \text{range}(\mathbf{A})$.

Now define a process indexed by $\mathbf{x} \in \mathbb{R}^n$ via composition:

$$h(\mathbf{x}) := \tilde{g}(\mathbf{Ax}), \quad \mathbf{x} \in \mathbb{R}^n.$$

For any finite set $\mathbf{x}_1, \dots, \mathbf{x}_m \in \mathbb{R}^n$, the vector $(h(\mathbf{x}_1), \dots, h(\mathbf{x}_m))$ is centered Gaussian with covariance

$$\text{Cov}(h(\mathbf{x}_i), h(\mathbf{x}_j)) = \text{Cov}(\tilde{g}(\mathbf{Ax}_i), \tilde{g}(\mathbf{Ax}_j)) = \tilde{k}(\mathbf{Ax}_i, \mathbf{Ax}_j) = k(\mathbf{x}_i, \mathbf{x}_j),$$

where the last equality is (41). Hence the finite-dimensional distributions of h match those of g (both centered Gaussian with identical covariance matrices), implying

$$g(\mathbf{x}) \stackrel{d}{=} h(\mathbf{x}) = \tilde{g}(\mathbf{Ax}) \quad \text{as stochastic processes on } \mathbb{R}^n.$$

This is statement (1).

The argument above already shows the claimed property: whenever (41) holds, $\tilde{k}$ is positive semi-definite on $\text{range}(\mathbf{A}) = \{\mathbf{Ax} : \mathbf{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^r$ and therefore defines a valid GP on that index set. $\square$

A.5. Amortized variational inference: alleviating the overconfidence issue

In this section, we explain why amortized variational inference can alleviate the overconfidence issue commonly observed in variational Bayesian models, including sparse variational GPs and Bayesian neural networks, from the perspective of the evidence lower bound (ELBO).

Overconfidence as a serious concern in Bayesian learning, especially in DKL, has been rigorously investigated in previous work. Consider a latent variable model with latent variables z and observations $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$. Variational inference seeks to approximate the true posterior $p(z \mid \mathcal{D})$ by a variational distribution $q(z)$ via the ELBO:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q(z)}[\log p(\mathcal{D} \mid z)] - \text{KL}(q(z) \parallel p(z)). \quad (42)$$

Overconfidence in standard variational inference: In standard (non-amortized) variational inference, the variational parameters of $q(z)$ are optimized independently for a given dataset. When expressive likelihood models are used (e.g., deep networks or flexible kernels), the ELBO objective admits solutions where:

- the likelihood term strongly favors sharp posterior modes that fit the data closely;
- the KL regularizer is insufficient to prevent posterior collapse toward overly concentrated distributions, especially under mean-field assumptions.

As a result, the learned posterior $q(z)$ may significantly underestimate posterior uncertainty, leading to overconfident predictions, particularly under distribution shift or in regions with limited data support. From an ELBO perspective, this phenomenon arises because the optimization is performed *pointwise* in the variational parameters: the posterior is allowed to specialize aggressively to the training data without being constrained to generalize across inputs.

Amortized variational inference introduces a shared inference model $q_\phi(z \mid x)$, parameterized by ϕ , typically implemented as a neural network. The ELBO becomes

$$\mathcal{L}_{\text{A-ELBO}} = \sum_{n=1}^N \mathbb{E}_{q_\phi(z \mid x_n)}[\log p(y_n \mid x_n, z)] - \text{KL}(q_\phi(z \mid x_n) \parallel p(z)). \quad (43)$$

Why amortized variational inference mitigates overconfidence: Crucially, amortization ties the variational parameters across all data points through shared mapping ϕ . This coupling introduces an implicit regularization effect:

- The variational posterior can no longer overfit individual data points independently;
- Sharp posterior concentrations must be explainable by a smooth inference function $q_\phi(z \mid x)$ that generalizes across inputs;
- Excessively confident posteriors that are inconsistent across nearby inputs are penalized indirectly, as they increase the overall ELBO loss.

This regularizing effect is particularly beneficial for uncertainty quantification. Under amortized inference, epistemic uncertainty is shaped not only by the local likelihood fit, but also by how well uncertainty patterns can be represented by the shared inference network. As a result, uncertainty is naturally preserved in out-of-distribution or data-sparse regions, where confident predictions would require extrapolations unsupported by the amortized posterior.

In the context of sparse variational GPs or their Bayesian network interpretations (Appendix A.3), amortized inference over low-dimensional latent variables (e.g., inducing weights or kernel coefficients) further strengthens this effect by limiting the capacity of the posterior to overfit. This provides a theoretically guided explanation for why amortized variational inference often yields better-calibrated uncertainty compared to non-amortized or pointwise variational approaches.

B. Experimental Details

B.1. Intuitive example: LoRA regression

Goal. We construct a 1D regression toy problem to visualize posterior mean and uncertainty after fine-tuning with Bayesian LoRA variants. The key diagnostic is the empirical coverage of nominal confidence intervals under an input-domain shift:

overconfident methods yield narrow intervals with low coverage, while well-calibrated methods maintain appropriate interval width in regions with limited evidence.

Data generation. Inputs are one-dimensional, $x \in \mathbb{R}$. We sample a latent function $f(\cdot)$ from a Gaussian process,

$$f \sim \mathcal{GP}(0, k_{\text{RBF}}(x, x')), \quad k_{\text{RBF}}(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right), \quad (44)$$

and generate observations with homoscedastic noise

$$y = f(x) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma_n^2). \quad (45)$$

Unless stated otherwise, we use $\ell = 1.0$, $\sigma_f = 1.0$, and $\sigma_n = 0.2$. We fix one GP draw f for the entire experiment (shared across all methods) to ensure comparability.

Pretraining vs fine-tuning splits. We create two domains to mimic a distribution shift.

Source (pretraining) domain: $x \sim \text{Unif}[-3, 3]$. We draw $N_{\text{src}} = 200$ training points and $N_{\text{src}}^{\text{test}} = 400$ test points from the same range. These correspond to the black (train) and red (test) points in Fig. 5(a).

Target (fine-tuning) domain: $x \sim \text{Unif}[-5, 5]$ (extrapolation / covariate shift). We draw $N_{\text{tgt}} = 200$ fine-tuning training points and $N_{\text{tgt}}^{\text{test}} = 400$ fine-tuning test points from this shifted support, corresponding to the blue (FT train) and green (FT test) points in Fig. 5(a).

All inputs are standardized using the source training statistics: $\tilde{x} = (x - \mu_{\text{src}}) / s_{\text{src}}$. Targets are kept in the original scale for visualization.

Backbone (pretrained) model. The pretrained predictor is a two-layer MLP with hidden width $d = 64$ and activation $\phi(\cdot) = \text{ReLU}$. The backbone is pretrained on the source domain by minimizing MSE with Adam (learning rate 10^{-3} , batch size 64, 5,000 steps). After pretraining, *all backbone weights are frozen* and only LoRA (or Bayesian LoRA) parameters are updated during fine-tuning.

LoRA parameterization. We attach LoRA to all linear layers (including the output head) for clarity in the 1D visualization. Let the frozen weight be $W_i \in \mathbb{R}^{1 \times d}$. LoRA introduces a low-rank update

$$W'_i = W_i + \Delta W_i, \quad \Delta W_i = B_i A_i, \quad i \in \{1, 2\}, \quad (46)$$

where $A \in \mathbb{R}^{r \times d_i}$ and $B \in \mathbb{R}^{d_{i+1} \times r}$, and rank $r = 2$ in all toy experiments. During fine-tuning, we optimize only the LoRA parameters; the backbone remains fixed.

BLoB: a Bayesian LoRA baseline where the adapted parameters follow a mean-field variational posterior (diagonal Gaussian) and are trained by maximizing an ELBO on the target training set. Predictive uncertainty is obtained by Monte Carlo sampling of LoRA parameters at test time (e.g., $T = 50$ samples).

C-LoRA: a contextual Bayesian LoRA variant where the variational parameters are produced by a lightweight amortization network conditioned on the layer input (or a summary of it), still under a mean-field posterior family. In the toy setup, the context network is a single hidden-layer MLP with tanh activation.

GPan-LoRA (ours): we model the Bayesian uncertainty in the low-rank adapted space via a GP-style correlated posterior over the rank- r coordinates, trained with amortized variational inference. Concretely, we parameterize the variational distribution with structured covariance in the r -dimensional subspace (rather than a fully factorized posterior in the ambient dimension), enabling calibrated uncertainty while retaining scalability. We use 32 inducing variables in the r -space and optimize the variational objective on the target training set.

Fine-tuning protocol (all Bayesian methods). All Bayesian LoRA methods are fine-tuned on the target training set for 2,000 steps using Adam (learning rate 5×10^{-4} , batch size 64). We apply KL warm-up on the ELBO term: the KL weight β increases linearly from 0 to 1 during the first 30% of updates, and remains 1 afterwards. For MC-based predictive estimates, we use $T = 50$ posterior samples for BLoB and C-LoRA; GPan-LoRA uses its closed-form / GP-based predictive where applicable, otherwise also uses $T = 1000$ samples for fair plotting.

GP ground-truth posterior (oracle). To provide the reference uncertainty in Fig. 5(c), we compute the exact GP posterior under the same kernel used to generate f (RBF with known hyperparameters). Given training data (X, y) , the posterior

predictive at x_* is

$$p(y_* | x_*, X, y) = \mathcal{N}(\mu_*(x_*), \sigma_*^2(x_*)), \quad (47)$$

with standard closed-form expressions. The plotted 95% CI is $\mu_* \pm 1.96 \sigma_*$.

Evaluation metrics. We report RMSE and NLL on both the source test set and the target test set, and emphasize calibration via empirical coverage of the nominal 95% CI:

$$\text{Cov}_{95} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[y_i \in [\mu(x_i) - 1.96\sigma(x_i), \mu(x_i) + 1.96\sigma(x_i)]]. \quad (48)$$

Overconfidence is reflected by $\text{Cov}_{95} \ll 0.95$ (intervals too narrow), which is the behavior observed for BLoB and C-LoRA in Fig. 5(d,e), while GPan-LoRA maintains coverage closer to the nominal level (Fig. 5(f)).

Plotting details. All curves are evaluated on a dense grid of x over $[-5, 5]$ (e.g., 400 evenly spaced points). We visualize the posterior mean (red line) and the 95% predictive interval (shaded region), and overlay training points as black dots for the corresponding domain in each panel. We use the same random seed, dataset realization, and pretrained backbone across all compared methods.

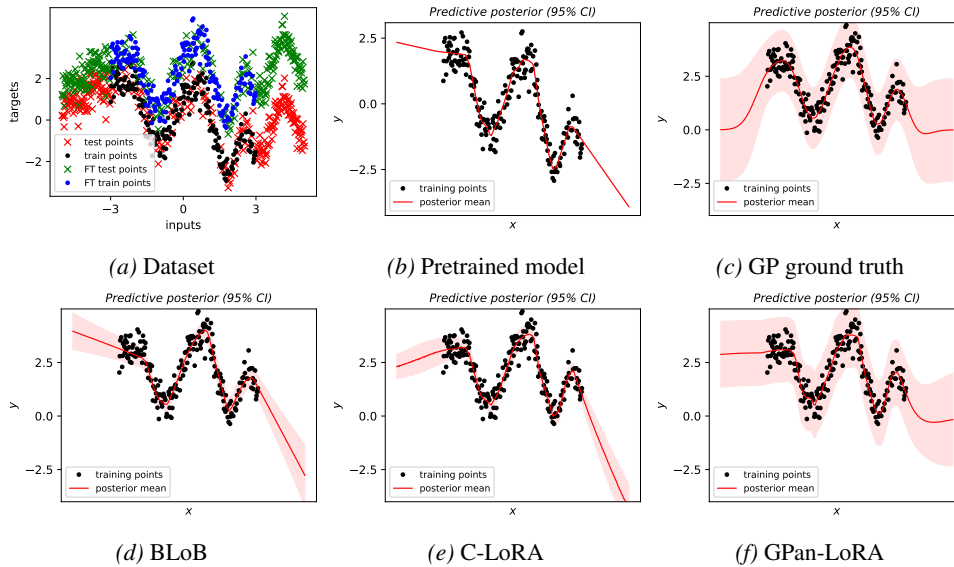


Figure 5. Fine-tuning a regression dataset with Bayesian LoRA. BLoB and C-LoRA exhibit low empirical coverage of confidence intervals, indicating overconfidence, while GPan-LoRA produces well-calibrated uncertainty estimates.

Table 4. MSE and NLL for fine-tuning regression datasets (ID and OOD) with Bayesian LoRA. Lower is better.

Method	In-distribution		Out-of-distribution	
	MSE ↓	NLL ↓	MSE ↓	NLL ↓
BLoB	0.21 ± 0.00	0.64 ± 0.11	2.48 ± 0.07	10.34 ± 1.82
C-LoRA	0.19 ± 0.00	0.78 ± 0.01	6.32 ± 0.05	6.77 ± 0.06
GPan-LoRA	0.19 ± 0.00	-0.11 ± 0.00	3.50 ± 0.00	1.98 ± 0.00

B.2. Implementation Details

Dataset. We summarize the downstream datasets and the corresponding prompt templates used for fine-tuning in Table 6. All tasks are formulated as multiple-choice or binary classification problems using unified, instruction-style prompts to ensure a consistent training and evaluation protocol across datasets.

Table 5. Hyperparameter settings for LLaMA3.1-8B and Qwen3-8B experiments.

Hyperparameter	LLaMA-3.1-8B	Qwen-3-8B
Max Seq. Length	300	512
LoRA Rank (r)	8	8
LoRA Alpha (α)	16	16
Target Modules	W_q, W_v W_{lm}	W_q, W_k, W_v $W_{gate}, W_{up}, W_{down}$
Training Iterations	5,000	
Batch Size	4	
Eval Interval	500	

Table 7 reports the size of the training split and the label space for each dataset. The considered benchmarks span a diverse range of commonsense reasoning settings, including binary (e.g., BoolQ, Winogrande) and multi-class classification (e.g., ARC, OBQA, MMLU), enabling a comprehensive evaluation of model performance and uncertainty behavior under varying label cardinalities and data scales.

Table 6. Prompts for common-sense reasoning tasks used for fine-tuning

Downstream task	Prompt template
Winogrande (WG-S/WG-M)	Select one of the choices that answers the following question: {question} Choices: A. {option1}. B. {option2}. Answer:
ARC (ARC-C/ARC-E), Openbook QA (OBQA), MMLU	Select one of the choices that answers the following question: {question} Choices: A. {choice1}. B. {choice2}, C. {choice3}. D. {choice4}. Answer:
BoolQ	Answer the question with only True or False: {question} Context: {passage}

Table 7. Training set size and label space size for each dataset

	WG-S	ARC-C	ARC-E	WG-M	OBQA	BoolQ
Number of training samples	512	892	1.8K	2.05k	3.97k	1.99k
Label space size	2	5	5	2	4	2

Uncertainty Quantification Evaluation Metrics. The quality of uncertainty quantification is commonly assessed using negative log-likelihood (NLL) and expected calibration error (ECE), which respectively capture model overconfidence and the alignment between predictive confidence and accuracy. For a model P_θ , the NLL over a dataset of size N is defined as

$$\text{NLL} = \frac{1}{N} \sum_{i=1}^N -\log P_\theta(y_i | x_i), \quad (49)$$

which penalizes low predictive probability assigned to the correct label y_i . In particular, overconfident but incorrect predictions incur a large penalty, leading to an increased NLL. In addition to NLL, ECE measures model calibration by quantifying the discrepancy between confidence and accuracy. Predictions are first grouped into M bins $\{B_m\}_{m=1}^M$ based on their confidence levels, and ECE is computed as

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)|, \quad (50)$$

where $\text{acc}(B_m)$ and $\text{conf}(B_m)$ denote the average accuracy and confidence within bin B_m , respectively:

$$\text{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbf{1}(\hat{y}_i = y_i), \quad \text{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} P_\theta(\hat{y}_i | x_i). \quad (51)$$

Base Models and Sequence Length. We conduct experiments using two base models LLaMA3.1-8B and Qwen3-8B. To account for architectural and tokenizer differences between the two models, we adopt model-specific maximum sequence lengths, setting the maximum sequence length to 300 for LLaMA3.1-8B and 512 for Qwen3-8B.

LoRA Injection. We apply LoRA adapters to LLaMA3.1-8B and Qwen3-8B with model-specific injection configurations. For LLaMA3.1-8B, LoRA adapters are applied to the query and value projection layers in the self-attention module, as well as to the final language modeling head that maps hidden states to vocabulary logits. For Qwen3-8B, LoRA adapters are applied to all linear projection layers within each Transformer block. This includes the query, key, value, and output projections in the self-attention module, as well as the gate, up, and down projections in the feed-forward network.

Baseline Implementations. We compare GPan-LoRA with several deterministic and Bayesian LoRA baselines, including MLE, Monte Carlo Dropout (MCD), Deep Ensembles, Laplace-LoRA (LAP), BLoB, and C-LoRA. Except for LAP, all baselines are implemented using the unified Bayesian-PEFT framework², ensuring consistent model architectures, data preprocessing, optimization settings, and evaluation protocols across methods. Note that for BLoB, C-LoRA, and GPan-LoRA, we draw one posterior sample per forward pass during training, and report results by averaging predictions over 10 posterior samples at evaluation. Deep Ensembles are implemented following prior work, using an ensemble size of $N = 3$ independently trained LoRA models with different random seeds, whose predictions are aggregated at inference time. Monte Carlo Dropout (MCD) is implemented by enabling dropout layers during inference and averaging predictions over multiple stochastic forward passes. Laplace-LoRA is implemented using the official Laplace-LoRA repository³, employing a Laplace approximation with a Kronecker-factored log-likelihood Hessian to approximate the posterior precision. The approximation is applied to all trainable parameters, following the default settings of the official implementation. As Laplace-LoRA is applied as a post-hoc Bayesian approximation on a pretrained deterministic LoRA model, its number of trainable parameters matches that of deterministic LoRA. C-LoRA is implemented using its official repository⁴, following the authors’ recommended configurations. For BLoB, we apply gradient clipping to stabilize training and prevent posterior collapse on small-scale datasets with limited samples. All remaining baseline configurations strictly follow their respective official implementations.

Training Protocol. All models are trained for 5,000 iterations with a batch size of 4. We evaluate model performance on a held-out validation set every 500 iterations. To reduce potential bias from overfitting—which is particularly important for uncertainty quantification—we select the checkpoint with the highest validation accuracy. Each experiment is repeated three times with different random seeds, and we report the mean and standard deviation of the results.

Bayesian LoRA Variants. For variational inference based Bayesian LoRA methods, including C-LoRA, BLoB, and our proposed **GPan-LoRA**, the coefficient of the KL divergence term significantly affects training stability. We tune this coefficient on the validation set over the candidate set $\{1, 5e^{-1}, 1e^{-1}, 5e^{-2}, 1e^{-2}, 5e^{-3}, 1e^{-3}\}$. We report results using the coefficient that achieves the best ECE on the validation set. Table 5 summarizes the specific hyperparameters used in our experiments.

B.3. Visualization Details

Visualization Protocol. Similar to our previous out-of-distribution (OOD) evaluation, we select 20 samples each from one in-distribution dataset, *OBQA*, and four OOD datasets: *ARC-E* and *ARC-C*, and *Chem* and *Phy*. For each input sample, we perform 50 stochastic forward passes to sample the latent embeddings from the Bayesian posterior. We then apply PCA (Jolliffe & Cadima, 2016) and t-SNE (Maaten & Hinton, 2008) for dimensionality reduction sequentially. The uncertainty of each sample is represented by the variance of its sampled embeddings, visualized as shaded covariance ellipses in Figure 4.

C. Additional Experiments

C.1. Computational Efficiency Analysis

Table 8 reports the model size and peak GPU memory consumption of the proposed methods and baselines during training on the WG-S dataset. All experiments were conducted on a single NVIDIA H200 GPU. Laplace approximation is applied *post hoc* to a trained deterministic LoRA model; since it does not introduce additional trainable parameters, we report the same number of trainable parameters as deterministic LoRA (MLE). Although LAP is a competitive baseline, its computational

²<https://github.com/Wang-ML-Lab/bayesian-peft>

³<https://github.com/adamxyang/laplace-lora>

⁴https://github.com/ahra99/c_lora

Table 8. Computational efficiency comparison. Values in parentheses denote the relative overhead w.r.t. MLE.

Method	Trainable Params (M)	Peak Memory (MB)	Inference Time (s)
MLE	4.47 ($\times 1.00$)	11,324 ($\times 1.00$)	0.093 ($\times 1.00$)
Deep Ensemble	13.40 ($\times 3.00$)	11,324 ($\times 1.00$)	2.812 ($\times 30.24$)
Laplace (LAP)	4.47 ($\times 1.00$)	33,121 ($\times 2.93$)	0.528 ($\times 5.68$)
BLoB	6.60 ($\times 1.48$)	11,868 ($\times 1.05$)	1.193 ($\times 12.83$)
C-LoRA	5.04 ($\times 1.13$)	11,280 ($\times 1.00$)	1.210 ($\times 13.01$)
GPan-LoRA	8.33 ($\times 1.86$)	11,363 ($\times 1.00$)	1.901 ($\times 20.44$)
LL GPan-LoRA	4.53 ($\times 1.01$)	11,337 ($\times 1.00$)	0.935 ($\times 10.05$)

Table 9. **Kernel sensitivity analysis of GPan-LoRA on LLaMA3.1-8B pre-trained weights.** We compare the default squared exponential (SE) kernel with a Matérn kernel under the same experimental protocol. Accuracy (ACC) and Expected Calibration Error (ECE) are reported in percentages, where “ $\uparrow$ ” and “ $\downarrow$ ” indicate that higher and lower values are preferred, respectively. The left six columns correspond to the in-distribution setting, while the right four columns report performance under distribution shift. The results indicate that GPan-LoRA is relatively robust to the kernel choice.

Metric	Method	In-Distribution Datasets						Out-of-Distribution Datasets (OBQA $\rightarrow$ X)			
								Small Dist. Shift		Large Dist. Shift	
		WG-S	ARC-C	ARC-E	WG-M	OBQA	BoolQ	ARC-C	ARC-E	Chem	Phy
ACC ($\uparrow$)	GPan-LoRA (SE)	72.41 $\pm$ 0.24	80.06 $\pm$ 1.01	91.84 $\pm$ 0.61	79.82 $\pm$ 0.44	86.66 $\pm$ 0.94	87.20 $\pm$ 3.29	75.78 $\pm$ 1.36	82.74 $\pm$ 1.23	48.00 $\pm$ 2.00	42.33 $\pm$ 0.57
	GPan-LoRA (Matérn)	74.13 $\pm$ 2.61	81.19 $\pm$ 1.86	91.96 $\pm$ 0.66	78.08 $\pm$ 0.70	85.53 $\pm$ 0.75	88.63 $\pm$ 0.49	76.23 $\pm$ 2.44	83.21 $\pm$ 0.56	48.66 $\pm$ 4.50	39.33 $\pm$ 2.08
ECE ($\downarrow$)	GPan-LoRA (SE)	13.39 $\pm$ 2.67	4.60 $\pm$ 0.88	2.36 $\pm$ 0.85	6.85 $\pm$ 1.44	3.12 $\pm$ 0.64	1.45 $\pm$ 0.62	6.46 $\pm$ 0.87	3.31 $\pm$ 0.37	11.31 $\pm$ 2.75	13.29 $\pm$ 1.20
	GPan-LoRA (Matérn)	12.82 $\pm$ 3.20	5.48 $\pm$ 1.70	2.76 $\pm$ 0.82	5.00 $\pm$ 0.61	2.77 $\pm$ 0.17	1.38 $\pm$ 0.38	6.18 $\pm$ 2.19	3.62 $\pm$ 0.43	12.58 $\pm$ 2.40	17.76 $\pm$ 2.49
NLL ($\downarrow$)	GPan-LoRA (SE)	0.69 $\pm$ 0.08	0.51 $\pm$ 0.02	0.24 $\pm$ 0.01	0.48 $\pm$ 0.01	0.37 $\pm$ 0.02	0.30 $\pm$ 0.06	0.60 $\pm$ 0.03	0.46 $\pm$ 0.01	1.19 $\pm$ 0.01	1.25 $\pm$ 0.01
	GPan-LoRA (Matérn)	0.72 $\pm$ 0.11	0.49 $\pm$ 0.02	0.23 $\pm$ 0.01	0.48 $\pm$ 0.01	0.38 $\pm$ 0.04	0.27 $\pm$ 0.01	0.58 $\pm$ 0.01	0.45 $\pm$ 0.02	1.19 $\pm$ 0.05	1.25 $\pm$ 0.01

overhead is substantially higher than that of other methods, including GPan-LoRA. Among variational inference-based approaches, C-LoRA and GPan-LoRA exhibit the smallest model sizes. This result highlights that GPan-LoRA achieves competitive memory efficiency while retaining a lightweight parameterization.

C.2. Kernel Sensitivity Analysis

Our main experiments use the squared exponential (SE) kernel as the default kernel choice in GPan-LoRA. However, the proposed framework is not restricted to a specific kernel form: the low-rank GP formulation can incorporate any positive semi-definite kernel that is well-defined in the projected LoRA feature space. To examine whether the empirical performance is sensitive to this choice, we conduct an additional kernel ablation by replacing the SE kernel with a Matérn kernel while keeping the remaining training and evaluation settings unchanged.

Table 9 reports the results on both in-distribution datasets and distribution-shifted OOD datasets. Overall, the Matérn kernel yields performance comparable to the SE kernel across ACC, ECE, and NLL. In several in-distribution settings, Matérn slightly improves ACC or calibration, while SE remains stronger on some large-shift OOD calibration metrics such as Phy ECE. Importantly, the differences are generally modest, suggesting that the uncertainty benefits of GPan-LoRA mainly come from the low-rank GP Bayesian formulation and amortized variational inference rather than from a carefully tuned kernel choice. This supports the kernel-agnostic nature of our framework and indicates that GPan-LoRA is relatively robust to reasonable kernel choices.

C.3. Seed-Level Paired Significance Tests

To strengthen the empirical evidence beyond mean $\pm$ std summaries, we further conduct seed-level paired t -tests using the raw per-seed results reported in Table 2. Since the primary goal of GPan-LoRA is uncertainty quantification rather than maximizing accuracy, we focus this statistical analysis on calibration-oriented metrics, namely ECE and NLL. Accuracy results are reported in Table 2 to demonstrate competitive task performance.

For each baseline, metric, dataset, and random seed, we compute the paired improvement between GPan-LoRA and the baseline under the same evaluation setting. For ECE and NLL, where lower values are preferred, we define

$$\Delta = \text{Metric}_{\text{baseline}} - \text{Metric}_{\text{GPan}}. \quad (52)$$

Table 10. Seed-level paired t -test statistics comparing GPan-LoRA with baselines on uncertainty-oriented metrics under distribution shift. The ‘OOD’ group follows the settings adopted in Table 2 and includes OBQA $\rightarrow$ {ARC-C, ARC-E, Chem, Phy}; the ‘Large Dist. Shift’ group includes OBQA $\rightarrow$ {Chem, Phy}. Each paired observation corresponds to one dataset–seed pair. For ECE and NLL, improvement is defined as $\Delta = \text{Metric}_{\text{baseline}} - \text{Metric}_{\text{GPan}}$, where positive Δ indicates better performance of GPan-LoRA. ‘Win/n’ denotes the number of dataset–seed pairs where GPan-LoRA improves over the baseline. We report the mean paired improvement, 95% confidence interval, and Holm-corrected p -value.

Metric	Comparison	Out-of-Distribution Datasets					Large Dist. Shift Datasets				
		n	Win/n	Mean Δ	95% CI	p_{Holm}	n	Win/n	Mean Δ	95% CI	p_{Holm}
ECE	GPan-LoRA vs. MLE	12	12/12	+16.82	[+11.17, +22.48]	< .001	6	6/6	+24.80	[+20.11, +29.50]	< .001
	GPan-LoRA vs. LAP	12	6/12	+0.67	[−0.80, +2.14]	.340	6	5/6	+1.99	[−0.31, +4.30]	.077
	GPan-LoRA vs. BLoB	12	12/12	+8.96	[+3.85, +14.08]	.005	6	6/6	+16.00	[+11.11, +20.89]	.001
	GPan-LoRA vs. C-LoRA	12	12/12	+6.52	[+3.31, +9.73]	.003	6	6/6	+10.23	[+5.43, +15.04]	.006
NLL	GPan-LoRA vs. MLE	12	12/12	+0.68	[+0.47, +0.89]	< .001	6	6/6	+0.95	[+0.70, +1.20]	< .001
	GPan-LoRA vs. LAP	12	6/12	−0.02	[−0.08, +0.04]	.452	6	6/6	+0.06	[+0.02, +0.10]	.012
	GPan-LoRA vs. BLoB	12	10/12	+0.29	[+0.10, +0.47]	.017	6	6/6	+0.54	[+0.38, +0.70]	.001
	GPan-LoRA vs. C-LoRA	12	9/12	+0.10	[+0.02, +0.18]	.040	6	6/6	+0.21	[+0.13, +0.29]	.002

Thus, positive Δ consistently indicates better performance of GPan-LoRA. This paired design compares methods under the same dataset and random seed, reducing variability due to different data splits or random initialization.

Because each individual dataset has only three random seeds, we avoid drawing per-dataset significance claims. Instead, we aggregate paired differences across dataset–seed pairs within each evaluation split. The Out-of-Distribution (OOD) analysis uses 12 paired observations from four OOD datasets and three random seeds, while the Large OOD analysis uses 6 paired observations from Chem and Phy across three random seeds. We apply two-sided paired t -tests and use Holm correction to account for multiple baseline comparisons within each metric and split.

As shown in Table 10, GPan-LoRA provides consistent improvements in uncertainty-oriented metrics under distribution shift. Compared with MLE, BLoB, and C-LoRA, GPan-LoRA significantly reduces both ECE and NLL on OOD datasets, with particularly strong improvements under large distribution shifts. Compared with LAP, the overall OOD differences are smaller and not uniformly significant, but GPan-LoRA achieves a consistent NLL improvement under large OOD shifts. These results support our main claim that GPan-LoRA improves calibration and probabilistic prediction quality under distribution shift, while avoiding over-claiming significance on raw accuracy.

C.4. Evaluation on Qwen3-8B

To examine whether the conclusions drawn in the main paper generalize beyond a single base model, we additionally evaluate all uncertainty estimation methods on Qwen3-8B pre-trained weights, using the same experimental protocol, datasets, and evaluation metrics as in Table 2.

As shown in Table 11, the overall trends are highly consistent with those observed on LLaMA3.1-8B. In particular, GPan-LoRA continues to demonstrate competitive predictive accuracy while achieving substantially improved calibration and negative log-likelihood, especially under distribution shift. These results suggest that the advantages of GPan-LoRA are not specific to a particular backbone, but hold across different large language model architectures.

Table 11. Performance of different uncertainty estimate methods applied to LoRA on Qwen3-8B pre-trained weights, where Accuracy (ACC) and Expected Calibration Error (ECE) are reported in percentages. “↑” and “↓” indicate that higher and lower values are preferred, respectively. The left six columns demonstrate the UQ performance of in-distribution setting, and the right four columns is the UQ performance under distribution shift. Best and second-best ECE and NLL results are highlighted in **boldface** and underline, respectively.

Metric	Method	In-Distribution Datasets						Out-of-Distribution Datasets (OBQA→X)			
								Small Dist. Shift		Large Dist. Shift	
		WG-S	ARC-C	ARC-E	WG-M	OBQA	BoolQ	ARC-C	ARC-E	Chem	Phy
ACC (↑)	MLE	73.91 $\pm$ 0.73	88.73 $\pm$ 0.85	96.59 $\pm$ 0.20	81.67 $\pm$ 0.29	89.46 $\pm$ 0.41	89.85 $\pm$ 0.22	88.28 $\pm$ 0.39	94.83 $\pm$ 0.10	55.66 $\pm$ 3.05	54.33 $\pm$ 4.04
	MCD	75.23 $\pm$ 0.89	87.61 $\pm$ 0.78	96.36 $\pm$ 0.20	82.46 $\pm$ 0.98	91.53 $\pm$ 0.23	90.00 $\pm$ 0.06	90.37 $\pm$ 0.23	95.42 $\pm$ 0.00	53.50 $\pm$ 2.12	58.00 $\pm$ 2.82
	DeepEns	74.55 $\pm$ 0.78	88.51 $\pm$ 1.21	96.71 $\pm$ 0.26	81.80 $\pm$ 0.49	90.46 $\pm$ 0.83	90.08 $\pm$ 0.17	89.86 $\pm$ 0.67	96.01 $\pm$ 0.10	56.66 $\pm$ 1.52	57.33 $\pm$ 2.10
	LAP	73.27 $\pm$ 0.43	90.96 $\pm$ 1.57	96.36 $\pm$ 0.27	79.79 $\pm$ 0.34	90.87 $\pm$ 0.42	89.20 $\pm$ 0.46	90.85 $\pm$ 0.68	95.88 $\pm$ 0.44	56.33 $\pm$ 1.15	56.54 $\pm$ 2.47
	BLoB	73.52 $\pm$ 0.71	88.06 $\pm$ 1.08	96.65 $\pm$ 0.07	82.08 $\pm$ 0.72	91.06 $\pm$ 0.30	89.95 $\pm$ 0.30	89.18 $\pm$ 0.47	95.95 $\pm$ 0.74	53.50 $\pm$ 2.12	60.50 $\pm$ 0.70
	C-LoRA	75.94 $\pm$ 0.71	87.95 $\pm$ 0.39	95.95 $\pm$ 0.24	79.37 $\pm$ 2.73	90.60 $\pm$ 0.34	90.08 $\pm$ 0.11	90.09 $\pm$ 0.85	95.59 $\pm$ 0.30	55.33 $\pm$ 1.52	59.00 $\pm$ 1.00
	<u>GPan-LoRA</u>	<u>75.29$\pm$1.91</u>	<u>85.47$\pm$2.86</u>	<u>95.89$\pm$0.44</u>	<u>80.77$\pm$0.82</u>	<u>89.86$\pm$0.23</u>	<u>89.84$\pm$0.15</u>	<u>86.03$\pm$0.70</u>	<u>93.72$\pm$0.96</u>	<u>57.33$\pm$0.57</u>	<u>58.00$\pm$2.00</u>
ECE (↓)	MLE	23.12 $\pm$ 1.74	6.20 $\pm$ 0.99	2.11 $\pm$ 0.26	13.37 $\pm$ 0.55	4.69 $\pm$ 0.55	4.00 $\pm$ 1.77	10.46 $\pm$ 0.87	4.42 $\pm$ 0.23	27.76 $\pm$ 5.02	33.64 $\pm$ 3.31
	MCD	18.16 $\pm$ 4.44	9.48 $\pm$ 0.20	2.22 $\pm$ 0.66	14.74 $\pm$ 1.66	7.00 $\pm$ 1.89	4.42 $\pm$ 4.52	8.29 $\pm$ 0.11	3.73 $\pm$ 0.37	31.07 $\pm$ 0.11	29.63 $\pm$ 0.53
	DeepEns	24.12 $\pm$ 1.06	9.68 $\pm$ 0.92	2.64 $\pm$ 0.22	15.45 $\pm$ 1.28	7.53 $\pm$ 0.35	3.68 $\pm$ 2.04	5.92 $\pm$ 0.64	2.13 $\pm$ 0.12	19.40 $\pm$ 0.25	24.28 $\pm$ 3.22
	LAP	4.95$\pm$0.77	3.61$\pm$0.83	<u>1.70$\pm$0.84</u>	2.90$\pm$1.10	2.59$\pm$0.43	<u>1.67$\pm$0.82</u>	3.09$\pm$0.57	<u>2.05$\pm$0.32</u>	<u>14.78$\pm$1.14</u>	18.95 $\pm$ 2.39
	BLoB	11.57 $\pm$ 0.15	<u>5.58$\pm$3.09</u>	<u>1.98$\pm$0.59</u>	11.74 $\pm$ 0.69	5.55 $\pm$ 0.48	6.76 $\pm$ 0.31	6.75 $\pm$ 1.43	2.37 $\pm$ 0.30	29.19 $\pm$ 4.00	27.49 $\pm$ 1.99
	C-LoRA	10.58 $\pm$ 3.69	6.68 $\pm$ 1.16	2.19 $\pm$ 0.40	7.84 $\pm$ 2.71	4.23 $\pm$ 1.61	2.08 $\pm$ 0.53	5.06 $\pm$ 2.05	1.84$\pm$0.80	18.51 $\pm$ 2.99	18.78$\pm$4.58
	<u>GPan-LoRA</u>	<u>13.37$\pm$1.38</u>	<u>5.82$\pm$1.05</u>	1.33$\pm$0.21	<u>4.68$\pm$1.65</u>	<u>2.86$\pm$0.39</u>	1.19$\pm$0.83	<u>5.03$\pm$1.06</u>	<u>2.74$\pm$0.60</u>	12.99$\pm$1.18	<u>18.86$\pm$3.45</u>
NLL (↓)	MLE	1.56 $\pm$ 0.47	0.33 $\pm$ 0.01	0.13 $\pm$ 0.02	0.63 $\pm$ 0.01	0.29 $\pm$ 0.01	0.27 $\pm$ 0.02	0.77 $\pm$ 0.06	0.33 $\pm$ 0.01	1.88 $\pm$ 0.13	1.75 $\pm$ 0.06
	MCD	1.11 $\pm$ 0.65	0.46 $\pm$ 0.04	0.13 $\pm$ 0.03	0.99 $\pm$ 0.33	0.49 $\pm$ 0.02	0.31 $\pm$ 1.04	0.61 $\pm$ 0.04	0.24 $\pm$ 0.03	1.91 $\pm$ 0.09	1.72 $\pm$ 0.06
	DeepEns	2.03 $\pm$ 0.22	0.77 $\pm$ 0.20	0.18 $\pm$ 0.1	0.94 $\pm$ 0.18	0.42 $\pm$ 0.01	0.26 $\pm$ 1.93	0.38 $\pm$ 0.01	0.16 $\pm$ 0.01	1.37 $\pm$ 0.04	1.26 $\pm$ 0.01
	LAP	0.54$\pm$0.01	0.30$\pm$0.04	0.10$\pm$0.01	0.44$\pm$0.01	0.24$\pm$0.01	<u>0.27$\pm$0.01</u>	0.28$\pm$0.01	0.11$\pm$0.01	1.24 $\pm$ 0.01	<u>1.10$\pm$0.01</u>
	BLoB	0.79 $\pm$ 0.05	0.49 $\pm$ 0.19	0.15 $\pm$ 0.01	0.78 $\pm$ 0.13	0.38 $\pm$ 0.01	0.32 $\pm$ 0.01	0.55 $\pm$ 0.01	0.18 $\pm$ 0.02	2.04 $\pm$ 0.01	1.87 $\pm$ 0.07
	C-LoRA	0.68 $\pm$ 0.08	0.40 $\pm$ 0.06	0.13 $\pm$ 0.01	0.50 $\pm$ 0.02	0.29 $\pm$ 0.03	0.24$\pm$0.01	0.32 $\pm$ 0.04	0.13 $\pm$ 0.02	1.21 $\pm$ 0.15	1.06$\pm$0.15
	<u>GPan-LoRA</u>	<u>0.69$\pm$0.07</u>	<u>0.47$\pm$0.02</u>	<u>0.12$\pm$0.01</u>	<u>0.46$\pm$0.02</u>	<u>0.28$\pm$0.01</u>	0.24$\pm$0.04	<u>0.41$\pm$0.01</u>	<u>0.18$\pm$0.01</u>	1.20$\pm$0.02	<u>1.06$\pm$0.04</u>